

TRIKE

Algorithm Specifications and Supporting Documentation (v1.5)

Tianrui Wang

Institute for Network Sciences and Cyberspace, Tsinghua University, Beijing, China

Yuxuan Li

Department of Computer Science and Engineering, Southern University of Science and Technology, Shenzhen, China

Jing Tian

School of Integrated Circuits, Nanjing University, Suzhou, China

Yangcan Zhou

School of Integrated Circuits, Sun Yat-sen University, Shenzhen, China

Zijian Zhang

School of Integrated Circuits, Nanjing University, Suzhou, China

Anyu Wang

Institute for Advanced Study, Tsinghua University, Beijing, China

Contact: Anyu Wang anyuwang@tsinghua.edu.cn

Table of Contents

1	Algorithm Specification	3
1.1	Properties of TRIKE	3
1.2	Notations	3
1.3	Security Definitions	4
1.4	Specification of TRIKE	5
1.4.1	Setup	5
1.4.2	Public Key Encryption	6
1.4.3	Key Encapsulation Mechanism	6
1.4.4	Decoder	7
1.5	Parameter Sets of TRIKE	9
2	Performance Analysis	10
2.1	Experimental Setup	10
2.2	Memory and Communication Bandwidth	10
2.3	Reference Implementation	12
2.4	Optimized Implementation	12
3	Decryption Failure Rate Analysis	13
3.1	Theoretical DFR Model	13
3.2	Validation of the FLS-Based DFR Model	14
3.3	Concrete DFR Estimate	14
3.4	Weak Key Rejection	15
4	Security Proofs	16
4.1	Security Assumptions and Hardness Reduction	16
4.2	OW-CPA Security	19
4.3	IND-CPA Security	21
4.4	IND-CCA Security	24
4.5	Security Proof with Weak Key Rejection	25
5	Security Against Known Attacks	26
5.1	Information Set Decoding	26
A	Details of the Weak Key Rejection	28
A.1	Type I	28
A.2	Type II	29
A.3	Type III	31
A.4	Gathering Property	31

1 Algorithm Specification

This document presents **TRIKE**, an IND-CCA2 secure key encapsulation mechanism (KEM) called Triple Quasi-Cyclic Code Key Encapsulation Mechanism.

TRIKE is a code based KEM leveraging the quasi-cyclic structure to reduce the public key size from $O(n^2)$ to $O(n)$. The security of TRIKE is based on the Quasi-Cyclic Syndrome Decoding (QCSD) problem and the Ratio Quasi-Cyclic Codeword Finding (RQCCF) problem, a new variant of the classical Syndrome Decoding (SD) problem that can be reduced to the Quasi-Cyclic Codeword Finding (QCCF) problem. Both the QCSD and QCCF problems are widely regarded as computationally hard problems.

HQC [1] and BIKE [2] were two candidates in the post-quantum cryptography (PQC) standardization process launched by the National Institute of Standards and Technology (NIST) [3]. Although BIKE has a much lower space cost than HQC, concerns regarding its decryption failure rate (DFR) analysis and vulnerability to weak key attacks prevented its selection. In contrast, HQC was recently selected in the fifth round of the NIST PQC standardization process.

Our construction TRIKE is conceptually related to prior schemes including BIKE and HQC, while concealing the sparse secret key by combining linear and division operations, resulting in security based on the RQCCF problem. With these new techniques, TRIKE achieves a much smaller code length and Hamming weight, which significantly reduces the decoding time cost and public key size. We also adopt a more accurate model for DFR prediction and reject weak keys that may introduce potential vulnerabilities. Compared to HQC, TRIKE reduces the bandwidth cost by about 40%.

The construction of TRIKE follows a two-stage approach: we first introduce an IND-CPA-secure public key encryption (PKE) scheme, and then apply the Fujisaki-Okamoto transformation with implicit rejection [4] to build the IND-CCA2-secure KEM.

This document is organized as follows: Section 1 presents the algorithm specification. Section 2 provides implementation details and performance results. Section 3 analyzes the decryption failure rate. Section 4 presents the security proofs, and Section 5 analyzes the concrete complexities of known attacks. Appendix A discusses the effect of weak key rejection.

1.1 Properties of TRIKE

To summarize, TRIKE is a KEM built upon a **novel triple quasi-cyclic framework** with a ratio-based mechanism for hiding the sparse secret structure. The security of TRIKE relies on QCSD and RQCCF, where RQCCF can be further reduced to the well-studied 3-QCCF problem, and both are widely believed to be computationally hard.

Compared to HQC, TRIKE achieves a **40%** reduction in bandwidth size by leveraging the quasi-cyclic structure. Compared to BIKE, it provides a more accurate DFR model and a rejection strategy for weak key instances, preventing decryption failure based attacks. In particular, all known weak keys are rejected with overwhelmingly high probability, making their contribution to the overall DFR negligible. These design choices lead to reduced code parameters in terms of length and Hamming weight, resulting in low communication cost and fast decoding, with the decoding time reduced to approximately **50%** compared to BIKE, while maintaining a provably secure reduction to standard code-based hardness assumptions.

1.2 Notations

The following notations will be used in this paper.

- Let $\mathbb{F}_q$ be the finite field with q elements, and $\mathbb{F}_q^n$ be the set of vectors where each element is in $\mathbb{F}_q$.
- For a vector $\mathbf{y} = (y_0, \dots, y_{n-1}) \in \mathbb{F}_2^n$, denote $w_H(\mathbf{y})$ to be the Hamming weight of $\mathbf{y}$, denote $\text{Supp}(\mathbf{y})$ to be the support of $\mathbf{y}$, and denote $\mathbf{y}^{[a,b]} := (y_a, y_{a+1}, \dots, y_{b-1})$, where the subscripts are taken mod n .
- Let $\star$ be the vector operator Schur product. For vector $\mathbf{u}, \mathbf{v} \in \mathbb{F}_2^n$, the result of $\mathbf{u} \star \mathbf{v}$ is defined as

$$\mathbf{u} \star \mathbf{v} = (u_0 \cdot v_0, u_1 \cdot v_1, \dots, u_{n-1} \cdot v_{n-1})$$

- Let $\mathcal{R} := \mathbb{F}_2[x]/(x^r - 1)$. An element in the ring $\mathcal{R}$ is represented in a polynomial form, i.e., $y = y_0 + y_1x + \dots + y_{r-1}x^{r-1}$, and the bold-case letter $\mathbf{y} = (y_0, \dots, y_{r-1})$ will be used to denote the coefficient vector of y . A circulant matrix corresponding the coefficients of y can be defined as

$$\text{rot}(\mathbf{y}) = \begin{pmatrix} y_0 & y_{r-1} & \dots & y_1 \\ y_1 & y_0 & \dots & y_2 \\ \vdots & \vdots & \ddots & \vdots \\ y_{r-1} & y_{r-2} & \dots & y_0 \end{pmatrix}. \quad (1)$$

Then for any $y, z \in \mathcal{R}$, the coefficient vector of yz equals to $\text{rot}(\mathbf{y}) \cdot \mathbf{z}$. Moreover, the i -th column of $\text{rot}(y)$ is denoted by $\text{Col}_i(y)$.

- Suppose i is co-prime to r , then the map

$$\phi_i : y(x) \rightarrow y(x^i)$$

defines an isomorphism of $\mathcal{R}$ to $\mathcal{R}$. Particularly, this isomorphism preserves the Hamming weight.

- To describe the spectrum of elements in $\mathcal{R}$, the distance between two positions $i, j \in \{0, \dots, r-1\}$ is defined as $d(i, j) = \min\{\pm(j - i) \bmod r\}$. For a polynomial $h \in \mathcal{R}$, the multiplicity of a distance δ counts the number of pairs of its nonzero coefficients separated by δ , which is formally expressed as $\mu(\delta, h) = |\{(i, j) \mid d(i, j) = \delta, h_i = h_j = 1, 0 \leq i \leq j < r\}|$. Then, the spectrum of h is defined as the set of all valid distances alongside their corresponding multiplicities, given by $\text{Spec}(h) = \{(\delta, \mu(\delta, h)) \mid \delta \in \{1, \dots, \lfloor r/2 \rfloor\}\}$.
- Let $\mathcal{R}_{\text{odd}} = \{y \in \mathcal{R} \mid w_H(y) \equiv 1 \pmod{2}\}$, $\mathcal{R}_{\text{even}} = \{y \in \mathcal{R} \mid w_H(y) \equiv 0 \pmod{2}\}$
- Denote
 - $\mathbf{K}(w) := \{(h_0, h_1, h_2) \in \mathcal{R}^3 \mid w_H(h_0) = w_H(h_1) = w_H(h_2) = w/3\}$,
 - $\mathbf{E}(t) := \{(e_0, e_1, e_2) \in \mathcal{R}^3 \mid w_H(e_0) + w_H(e_1) + w_H(e_2) = t\}$,

1.3 Security Definitions

In this paper, we employ four security notions: OW-CPA, OW-PCA and IND-CPA for PKE, and IND-CCA for KEM, respectively.

Definition 1 (One-Wayness under Chosen Plaintext Attacks (OW-CPA)). Let $\text{PKE} = (\text{KeyGen}, \text{Encrypt}, \text{Decrypt})$ be a public-key encryption scheme with message space $\mathcal{M}$, $\mathcal{P}_{\text{CO}}$ be a plaintext-checking oracle such that

$$\mathcal{P}_{\text{CO}}(m, c) = (\text{Decrypt}(\text{sk}, c) = m).$$

For a probabilistic polynomial-time (PPT) adversary $\mathcal{A}$, the OW-CPA game is defined as:

Game	$G_{\text{OW-CPA}}(\mathcal{A})$
	1. $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}()$ 2. $m^* \xleftarrow{\\$} \mathcal{M}$ 3. $c^* \leftarrow \text{Encrypt}(\text{pk}, m^*)$ 4. $m' \leftarrow \mathcal{A}(\text{pk}, c^*)$ 5. return $\mathcal{P}_{\text{CO}}(m', c^*)$

The advantage of the OW-CPA adversary $\mathcal{A}$ is defined as:

$$\text{Adv}_{\text{PKE}}^{\text{OW-CPA}}(\mathcal{A}) = \Pr[G_{\text{OW-CPA}}(\mathcal{A})].$$

Definition 2 (One-Wayness under Plaintext Checking Attacks (OW-PCA)). Let $\text{PKE} = (\text{KeyGen}, \text{Encrypt}, \text{Decrypt})$ be a public-key encryption scheme with message space $\mathcal{M}$, $\mathcal{P}_{\text{CO}}$ be a plaintext-checking oracle, as defined above.

For a PPT adversary $\mathcal{A}$ that is allowed to query the plaintext-checking oracle $\mathcal{P}_{\text{CO}}$, the OW-PCA game is defined as:

Game $G_{\text{OW-PCA}}(\mathcal{A})$
1. $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}()$ 2. $m^* \xleftarrow{\\$} \mathcal{M}$ 3. $c^* \leftarrow \text{Encrypt}(\text{pk}, m^*)$ 4. $m' \leftarrow \mathcal{A}^{\mathcal{P}_{\text{CO}}}(\text{pk}, c^*)$ 5. <i>return</i> $\mathcal{P}_{\text{CO}}(m', c^*)$

The advantage of the OW-PCA adversary $\mathcal{A}$ is defined as:

$$\text{Adv}_{\text{PKE}}^{\text{OW-PCA}}(\mathcal{A}) = \Pr[G_{\text{OW-PCA}}(\mathcal{A})].$$

Definition 3 (Indistinguishability under Chosen Plaintext Attacks (IND-CPA)). Let $\text{PKE} = (\text{KeyGen}, \text{Encrypt}, \text{Decrypt})$ be a public-key encryption scheme. For a PPT adversary $\mathcal{A} = (\mathcal{A}_{\text{choose}}, \mathcal{A}_{\text{guess}})$, the IND-CPA game is defined as:

Game $G_{\text{IND-CPA}}(\mathcal{A})$
1. $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}()$ 2. $(m_0, m_1) \leftarrow \mathcal{A}_{\text{choose}}(\text{pk})$ 3. $b \xleftarrow{\\$} \{0, 1\}$ 4. $c^* \leftarrow \text{Encrypt}(\text{pk}, m_b)$ 5. $b' \leftarrow \mathcal{A}_{\text{guess}}(\text{pk}, c^*)$ 6. <i>return</i> $(b = b')$

The advantage of the IND-CPA adversary $\mathcal{A}$ is defined as:

$$\text{Adv}_{\text{PKE}}^{\text{IND-CPA}}(\mathcal{A}) = \left| \Pr[G_{\text{IND-CPA}}(\mathcal{A})] - \frac{1}{2} \right|.$$

Definition 4 (Indistinguishability under Chosen Ciphertext Attacks (IND-CCA)¹). Let $\text{KEM} = (\text{KeyGen}, \text{Encaps}, \text{Decaps})$ be a key encapsulation mechanism. For a PPT adversary $\mathcal{A}$ with a decapsulation oracle $\mathcal{O}_{\text{Decaps}}(c)$ such that $c \neq c^*$, the IND-CCA game is defined as:

Game $G_{\text{IND-CCA}}(\mathcal{A})$
1. $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}()$ 2. $b \xleftarrow{\\$} \{0, 1\}$ 3. $(k_0, c^*) \leftarrow \text{Encaps}(\text{pk})$ 4. $k_1 \xleftarrow{\\$} \mathcal{M}$ 5. $b' \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Decaps}}}(k_b, c^*)$ 6. <i>return</i> $(b = b')$

The advantage of the IND-CCA adversary $\mathcal{A}$ is defined as:

$$\text{Adv}_{\text{KEM}}^{\text{IND-CCA}}(\mathcal{A}) = \left| \Pr[G_{\text{IND-CCA}}(\mathcal{A})] - \frac{1}{2} \right|.$$

1.4 Specification of TRIKE

1.4.1 Setup Given the security level λ as input, the setup phase outputs the system parameters $\{r, w, t, l\}$, the hash functions H_1, H_2, H_3, H_4, K, L , and a corresponding decoder.

¹ Although we refer to our security notion as IND-CCA, it is equivalent to the standard IND-CCA2 definition, since the decapsulation oracle remains accessible after the challenge ciphertext is given.

System Parameters

- r is the block length and we choose r as a prime number where 2 is primitive module r
- w is the total weight of secret vectors, where $d = w/3$ is an odd number
- t is the total weight of error vectors
- l is the length of seed and message

Hash Functions We define the seed or message space as $M = \{0, 1\}^l$. Then we select hash functions uniformly at random with the following domains and ranges.

- $H_1 : M \rightarrow \mathcal{R}_{even}$
- $H_2 : M \rightarrow \mathcal{R}_{even}$
- $H_3 : M \rightarrow \mathcal{R}_{odd}$
- $H_4 : M \times \mathcal{R}_{odd} \rightarrow E(t)$
- $K : M \times \mathcal{R} \times \mathcal{R} \rightarrow M$
- $L : E(t) \rightarrow M$

Decoder The TRIKE Bit-Flipping decoder is discussed in Section 1.4.4.

1.4.2 Public Key Encryption The formal procedures of PKE are defined in Algorithm 1, Algorithm 2 and Algorithm 3, respectively.

Algorithm 1: PKE.KeyGen

Input: None
Output: Public key pk and secret key sk

- 1: $h_0, h_1, h_2 \xleftarrow{\$} K(w), \sigma \xleftarrow{\$} M$
- 2: $t_1, t_2, r_1 \leftarrow H_1(\sigma), H_2(\sigma), H_3(\sigma)$
- 3: $t_0 \leftarrow \frac{r_1 h_0 + h_1}{r_1 + t_1}, r_2 \leftarrow \frac{h_2 + t_0 t_2}{t_0 + h_0}$
- 4: $pk \leftarrow (\sigma, r_2), sk \leftarrow (\sigma, h_0, h_1, h_2)$
- 5: **return** (pk, sk)

Algorithm 2: PKE.Encrypt

Input: Public key pk
Output: Ciphertext c

- 1: $(e_0, e_1, e_2) \xleftarrow{\$} E(t)$.
- 2: $t_1, t_2, r_1 \leftarrow H_1(\sigma), H_2(\sigma), H_3(\sigma)$
- 3: $u \leftarrow e_0 + e_1 r_1 + e_2 r_2$
- 4: $v \leftarrow e_0 + e_1 t_1 + e_2 t_2$
- 5: $c \leftarrow (u, v)$
- 6: **return** c

1.4.3 Key Encapsulation Mechanism The formal procedures of KEM are defined in Algorithm 4, Algorithm 5 and Algorithm 6.

- Setup(1^λ): Choose the parameters r, w, t, l .
- KeyGen(): Let $h_0, h_1, h_2 \xleftarrow{\$} K(w), \sigma \xleftarrow{\$} M, \sigma' \xleftarrow{\$} M, t_1 \leftarrow H_1(\sigma), t_2 \leftarrow H_2(\sigma)$, and $r_1 \leftarrow H_3(\sigma)$. Calculate $t_0 \leftarrow \frac{r_1 h_0 + h_1}{r_1 + t_1}$ and $r_2 \leftarrow \frac{h_2 + t_0 t_2}{t_0 + h_0}$. Set $pk = (\sigma, \sigma', r_2), sk = (\sigma, h_0, h_1, h_2)$ and output (pk, sk) .
- Encaps(pk): Let $m \xleftarrow{\$} \{0, 1\}^l, e_0, e_1, e_2 \leftarrow H_4(m, r_2)$. Calculate $u \leftarrow e_0 + e_1 r_1 + e_2 r_2, v \leftarrow e_0 + e_1 t_1 + e_2 t_2$. Output $c \leftarrow (u, v, m + L(e_0, e_1, e_2)), K \leftarrow K(m, c)$.

Algorithm 3: PKE.Decrypt**Input:** Secret key $\mathbf{sk}$ and ciphertext c **Output:** Error vector e'

- 1: $(u, v) \leftarrow c$
- 2: $t_1, r_1 \leftarrow \mathbf{H}_1(\sigma), \mathbf{H}_3(\sigma)$
- 3: $t_0 \leftarrow \frac{r_1 h_0 + h_1}{r_1 + t_1}$
- 4: $s \leftarrow (h_0 + t_0)u - t_0 v$
- 5: $e'_0, e'_1, e'_2 \leftarrow \text{Decoder}(h_0, h_1, h_2, s)$
- 6: **return** $e' = (e'_0, e'_1, e'_2)$

- Decaps($\mathbf{sk}, c$): Let $(u, v, c_2) \leftarrow c$. Calculate $t_0 \leftarrow \frac{r_1 h_0 + h_1}{r_1 + t_1}$, $r_2 \leftarrow \frac{h_2 + t_0 t_2}{t_0 + h_0}$, $s \leftarrow (h_0 + t_0)u - t_0 v$. Figure out $e'_0, e'_1, e'_2 \leftarrow \text{Decoder}(h_0, h_1, h_2, s)$. $m' \leftarrow c_2 + \mathbf{L}(e'_0, e'_1, e'_2)$. If $(e'_0, e'_1, e'_2) = \mathbf{H}_4(m', r_2)$, then $K \leftarrow \mathbf{K}(m', c)$, else $K \leftarrow \mathbf{K}(\sigma', c)$.

Algorithm 4: KEM.KeyGen**Input:** Security parameter 1^λ **Output:** Public key $\mathbf{pk}$ and secret key $\mathbf{sk}$

- 1: $h_0, h_1, h_2 \xleftarrow{\$} \mathbf{K}(w)$, $\sigma \xleftarrow{\$} \mathbf{M}$, $\sigma' \xleftarrow{\$} \mathbf{M}$
- 2: $t_1, t_2, r_1 \leftarrow \mathbf{H}_1(\sigma), \mathbf{H}_2(\sigma), \mathbf{H}_3(\sigma)$
- 3: $t_0 \leftarrow \frac{r_1 h_0 + h_1}{r_1 + t_1}$, $r_2 \leftarrow \frac{h_2 + t_0 t_2}{t_0 + h_0}$
- 4: $\mathbf{pk} \leftarrow (\sigma, r_2)$, $\mathbf{sk} \leftarrow (\sigma, \sigma', h_0, h_1, h_2)$
- 5: **return** $(\mathbf{pk}, \mathbf{sk})$

Algorithm 5: KEM.Encaps**Input:** Public key $\mathbf{pk}$ **Output:** Transformed key K and ciphertext c

- 1: $m \xleftarrow{\$} \mathbf{M}$
- 2: $e_0, e_1, e_2 \leftarrow \mathbf{H}_4(m, r_2)$
- 3: $t_1, t_2, r_1 \leftarrow \mathbf{H}_1(\sigma), \mathbf{H}_2(\sigma), \mathbf{H}_3(\sigma)$
- 4: $u \leftarrow e_0 + e_1 r_1 + e_2 r_2$
- 5: $v \leftarrow e_0 + e_1 t_1 + e_2 t_2$
- 6: $c \leftarrow (u, v, m + \mathbf{L}(e_0, e_1, e_2))$
- 7: $K \leftarrow \mathbf{K}(m, c)$
- 8: **return** K, c

1.4.4 Decoder We adopt a Bit-Flipping (BF) decoder derived from the BIKE-Flipping decoder and adapted to the structure of TRIKE. The decoder is formalized in Algorithm 8.

Given the secret key (h_0, h_1, h_2) and an input syndrome s , the decoder maintains an estimated error vector

$$e' = (e'_0, e'_1, e'_2)$$

and a current syndrome s' , initially set to s .

At each iteration, a threshold T is computed as a function of the initial syndrome weight $w_H(s)$, the current syndrome weight $w_H(s')$, and the iteration number. The threshold selection rule is discussed separately below.

For every position $i \in \{0, \dots, r-1\}$, the decoder evaluates a counter value for each circulant block. The counter corresponds to the number of unsatisfied parity-check equations involving the

Algorithm 6: KEM.Decaps**Input:** Secret key sk and ciphertext c **Output:** Transformed key K

```

1:  $(u, v, c_2) \leftarrow c$ 
2:  $t_1, t_2, r_1 \leftarrow H_1(\sigma), H_2(\sigma), H_3(\sigma)$ 
3:  $t_0 \leftarrow \frac{r_1 h_0 + h_1}{r_1 + t_1}, r_2 \leftarrow \frac{h_2 + t_0 t_2}{t_0 + h_0}$ 
4:  $s \leftarrow (h_0 + t_0)u - t_0 v$ 
5:  $e'_0, e'_1, e'_2 \leftarrow \text{Decoder}(h_0, h_1, h_2, s)$ 
6:  $m' \leftarrow c_2 + L(e'_0, e'_1, e'_2)$ 
7: if  $(e'_0, e'_1, e'_2) = H_4(m', r_2)$  then
8:    $K \leftarrow K(m', c)$ 
9: else
10:   $K \leftarrow K(\sigma', c)$ 
11: end if
12: return  $K$ 

```

Algorithm 7: Decryption**Input:** Secret key sk and ciphertext c **Output:** Transformed key K

```

1:  $(u, v, c_2) \leftarrow c$ 
2:  $t_0 \leftarrow \frac{r_1 h_0 + h_1}{r_1 + t_1}$ 
3:  $s \leftarrow (h_0 + t_0)u - t_0 v$ 
4:  $e'_0, e'_1, e'_2 \leftarrow \text{Decoder}(h_0, h_1, h_2, s)$ 
5:  $m' \leftarrow c_2 + L(e'_0, e'_1, e'_2)$ 
6: if  $(e'_0, e'_1, e'_2) = H_4(m, r_2)$  then
7:    $K \leftarrow K(m', c)$ 
8: else
9:    $K \leftarrow K(\sigma', c)$ 
10: end if
11: return  $K$ 

```

considered bit and is computed as $w_H(\text{col}_i(h_j) \star s')$, where $\text{col}_i(h_j)$ denotes the i -th column of the circulant matrix generated by h_j , and $\star$ denotes the Schur product.

Whenever the counter associated with a bit position is greater than or equal to the threshold T , the corresponding bit of the estimated error vector is flipped. This procedure is applied independently to all three circulant blocks h_0 , h_1 , and h_2 .

After all candidate flips have been processed, the syndrome is recomputed as

$$s' = h_0 e'_0 + h_1 e'_1 + h_2 e'_2.$$

The process is repeated for a fixed number of iterations $I_{\max}$. The final estimate e' is then returned as the decoding result.

Threshold Selection. The threshold used in Algorithm 8 depends on both the initial syndrome weight and the current syndrome weight. Let

$$w = w_H(s), \quad w' = w_H(s').$$

The threshold is defined as

$$T = \max(c_a w' + c_b, T'),$$

where

$$T' = \begin{cases} c_a w + c_b + \delta, & \text{it} = 1, \\ \frac{2(c_a w + c_b) + M}{3} + \delta, & \text{it} = 2, \\ \frac{(c_a w + c_b) + 2M}{3} + \delta, & \text{it} = 3, \\ M + \delta, & \text{it} \geq 4, \end{cases}$$

Algorithm 8: TRIKE Bit-Flipping Decoder

Input: Secret key (h_0, h_1, h_2) , syndrome s , maximum number of iterations $I_{\max}$, threshold coefficients c_a, c_b, δ
Output: Error vector e'

<pre> 1: Initialize $e' \leftarrow 0, s' = s$ 2: for $it = 1$ to $I_{\max}$ do 3: $T = \text{threshold}(w_H(s), w_H(s'), it, c_a, c_b, \delta)$ 4: for $i = 0$ to $r - 1$ do 5: if $w_H(\text{Col}_i(h_0) \star s') \geq T$ then 6: $e'_{0i} = e'_{0i} \oplus 1$ 7: end if 8: end for 9: for $i = 0$ to $r - 1$ do 10: if $w_H(\text{Col}_i(h_1) \star s') \geq T$ then 11: $e'_{1i} = e'_{1i} \oplus 1$ 12: end if 13: end for 14: for $i = 0$ to $r - 1$ do 15: if $w_H(\text{Col}_i(h_2) \star s') \geq T$ then 16: $e'_{2i} = e'_{2i} \oplus 1$ 17: end if 18: end for 19: $s' = h_0 e'_0 + h_1 e'_1 + h_2 e'_2$ 20: end for 21: return e' </pre>	<pre> threshold($w, w', it, c_a, c_b, \delta$) 1: $T_{\text{now}} \leftarrow c_a \cdot w' + c_b$ 2: $T_{\text{init}} \leftarrow c_a \cdot w + c_b$ 3: $M \leftarrow (d + 1)/2$ 4: if $it = 1$ then 5: $T' \leftarrow T_{\text{init}} + \delta$ 6: else if $it = 2$ then 7: $T' \leftarrow (2T_{\text{init}} + M)/3 + \delta$ 8: else if $it = 3$ then 9: $T' \leftarrow (T_{\text{init}} + 2M)/3 + \delta$ 10: else 11: $T' \leftarrow M + \delta$ 12: end if 13: return $\max(T_{\text{now}}, T')$ </pre>
---	--

and

$$M = \frac{d + 1}{2}.$$

The first term, $c_a w' + c_b$, provides an adaptive threshold based on the current syndrome weight, while T' is computed from the initial syndrome weight and varies with the iteration number. During the first three iterations, T' gradually transitions from $c_a w + c_b + \delta$ to $M + \delta$. From the fourth iteration onward, it remains constant. The decoder uses the larger of the two values as the threshold for the current iteration.

The coefficients c_a and c_b were initially chosen according to the affine-threshold methodology proposed by [5,6]. For a syndrome of weight S , the theoretical threshold can be expressed as a function $F_t(S)$, where t denotes the error weight. Since direct evaluation of $F_t(S)$ is impractical during decoding, it is approximated by an affine function

$$f_t(S) = aS + b.$$

The coefficients are obtained from a local linear approximation around the average syndrome weight S_{avg} ,

$$\begin{aligned} a &= F_t(S_{\text{avg}} + 1) - F_t(S_{\text{avg}}), \\ b &= F_t(S_{\text{avg}}) - aS_{\text{avg}}. \end{aligned}$$

This affine approximation provides a reference relationship between syndrome weight and threshold, which serves as the basis for selecting the coefficients c_a and c_b . In practice, the values suggested by the affine-threshold analysis were used only as initial reference points. The final parameters (c_a, c_b, δ) were obtained through empirical tuning to optimize decoding performance and minimize the decoding failure rate. The resulting parameter sets are reported in Table 2. The same parameters are subsequently used for the DFR estimation presented in Section 3.

1.5 Parameter Sets of TRIKE

We propose five parameter sets, TRIKE-1/2/3/5/7/9, targeting classical security levels of 128/128/192/256/384/512 bits and quantum security levels of 64/80/96/128/192/256 bits. Among them,

TRIKE-2/5/7/9 satisfy the requirements of the Institute of Commercial Cryptography Standards (ICCS) [7], while TRIKE-1/3/5 correspond to NIST security levels 1, 3 and 5. So we only provide the instances of TRIKE-2/5/7/9 in the submission. The detailed parameters are given in Table 1. In addition, the recommended threshold coefficients for the TRIKE Bit-Flipping Decoder are listed in Table 2.

Table 1. Suggested TRIKE parameters

Instance	Classical Security	Quantum Security	r	w	t	l	DFR
TRIKE-1	128	64	10,301	81	201	128	2^{-128}
TRIKE-2	128	80	15,581	105	263	256	2^{-128}
TRIKE-3	192	96	22,003	123	319	256	2^{-192}
TRIKE-5	256	128	35,363	165	429	256	2^{-256}
TRIKE-7	384	192	69,691	249	659	512	2^{-384}
TRIKE-9	512	256	114,043	333	877	512	2^{-512}

Table 2. Suggested parameters for TRIKE Bit-Flipping Decoder

Instance	c_a	c_b	δ
TRIKE-1	0.0032097305	12.68	3
TRIKE-2	0.0026330727	16.52	4
TRIKE-3	0.0022725837	16.76	5
TRIKE-5	0.0018298784	21.05	6
TRIKE-7	0.0013203507	25.52	6
TRIKE-9	0.0010658397	30.25	7

2 Performance Analysis

2.1 Experimental Setup

The benchmarking platform. The experiments were carried out on a server node equipped with an Intel® Xeon® Platinum 8362 processor (Ice Lake generation) operating at a base frequency of 2.80 GHz. The platform is configured with approximately 1.5 TB of RAM and a multi-level cache hierarchy, including 48 KB L1d cache, 32 KB L1i cache, 1,280 KB L2 cache per core, and 49,152 KB shared L3 cache. Hyper-threading and Turbo Boost were disabled during benchmarking to ensure stable cycle measurements.

OS and compilation. The implementations were written in C and built using CMake (version 4.2.3). The code was compiled with gcc (Ubuntu 11.4.0) in 64-bit mode, using the optimization flags “-O3”, “-fno”, and “-funroll-all-loops”. The implementation uses the vector-PCLMULQDQ instruction and AVX512 architecture extensions, and all experiments were conducted on a Linux (Ubuntu 22.04.1 LTS) operating system.

2.2 Memory and Communication Bandwidth

In our implementations, the private key has size $4 \cdot w + 3 \cdot \lceil r/8 \rceil + 2 \cdot \lceil l/8 \rceil$, while the public key has size $\lceil r/8 \rceil + \lceil l/8 \rceil$. The ciphertext has size $2 \cdot \lceil r/8 \rceil + \lceil l/8 \rceil$. The communication bandwidth is the

sum of the public key and the ciphertext, i.e., $3 \cdot \lceil r/8 \rceil + 2 \cdot \lceil l/8 \rceil$. The resulting private key, public key, ciphertext sizes, and bandwidth for different TRIKE instances are given in bytes in Table 3.

Table 3. TRIKE Private Key, Public Key and Ciphertext sizes (in Bytes)

Instance	TRIKE-1	TRIKE-2	TRIKE-3	TRIKE-5	TRIKE-7	TRIKE-9
Private Key	4,220	6,328	8,812	13,988	27,260	44,228
Public Key	1,304	1,980	2,783	4,453	8,776	14,320
Ciphertext	2,592	3,928	5,534	8,874	17,488	28,576
Bandwidth	3,896	5,908	8,317	13,327	26,264	42,896

More generally, TRIKE private keys can be represented either as sparse binary vectors, as lists of nonzero-bit positions, or by a seed from which the key is deterministically generated, providing different space-time trade-offs. For maximum decapsulation performance, both the reference and optimized implementations store all data required during decapsulation: h_0, h_1, h_2 as nonzero-bit positions, h_0, t_0, r_2 as sparse vectors, and the random seeds σ, σ' . This design avoids repeated arithmetical operations and eliminates conversions between sparse-vector and position-list representations during decapsulation.

Since different private-key representations lead to different storage requirements, we focus on public-key size, ciphertext size, and communication bandwidth when comparing TRIKE with other code-based KEMs. To provide a broader perspective on communication efficiency, we further compare TRIKE with representative NIST code-based schemes, including Classic McEliece [8], BIKE [2], and HQC [1], at NIST Security Levels 1, 3 and 5. The comparison focuses on public key size, ciphertext size, and communication bandwidth, as summarized in Table 4. TRIKE achieves the smallest public key size across all security levels while maintaining a bandwidth comparable to that of BIKE. Compared to the standardized HQC, TRIKE significantly reduces bandwidth requirements, consuming only 58%, 62% and 62% of HQC’s bandwidth at the NIST-1, NIST-3 and NIST-5 security levels, respectively.

Table 4. Comparison with NIST Code-Based Standards (in Bytes)

Security Level	Scheme	Public Key	Ciphertext	Bandwidth
Level 1	McEliece348864	261,120	96	261,216
	BIKE-1	1,541	1,573	3,114
	HQC-1	2,241	4,433	6,674
	TRIKE-1	1,304	2,592	3,896
Level 3	McEliece460896	524,160	156	524,316
	BIKE-1	3,083	3,115	6,198
	HQC-1	4,514	8,978	13,492
	TRIKE-1	2,783	5,534	8,317
Level 5	McEliece6960119	1,047,319	194	1,047,513
	BIKE-5	5,122	5,154	10,276
	HQC-5	7,237	14,421	21,658
	TRIKE-5	4,453	8,874	13,327

2.3 Reference Implementation

The reference implementation is written in ISO C and is designed to be constant-time. All five instances described in this work are supported. To ensure portability across different platforms, it does not rely on any architecture-specific instruction sets. The implementation adopts Karatsuba multiplication for polynomial multiplication, combined with a fast polynomial inversion method as described in [9]. The decoder is implemented using a fixed-iteration BP algorithm with fixed-point arithmetic, ensuring both predictable runtime and resistance against timing side-channel attacks. Performance in kiloCycles is listed in Table 5.

Table 5. Performances of the reference implementation of TRIKE (in kiloCycles)

Instance	TRIKE-1	TRIKE-2	TRIKE-3	TRIKE-5	TRIKE-7	TRIKE-9
KeyGen	84,373	97,929	295,854	796,720	2,633,920	3,305,743
Encaps	8,885	9,443	25,690	75,286	223,593	227,994
Decaps	34,394	61,782	106,066	236,072	694,266	1,385,318

2.4 Optimized Implementation

The optimized implementation targets mainstream 64-bit PC processors and provides optional acceleration using AVX512 and Intel vector-PCLMULQDQ instructions. Similar to the reference version, it supports all five instances described in this work.

For arithmetic on $\mathcal{R}$, both implementations use Karatsuba multiplication and the same fast polynomial inversion approach inspired by [9]. In the optimized version, further acceleration is achieved by following [10], where the recursion boundary of Karatsuba is carefully chosen at the 512×512 level, allowing efficient use of vector-PCLMULQDQ instructions for the base-case multiplications.

For the decoder, the same fixed-iteration fixed-point BP decoding algorithm is employed. However, the optimized implementation exploits AVX512 vectorization to process multiple messages in parallel, achieving an effect close to 8-way parallelism. This significantly reduces the overall decoding latency while preserving constant-time behavior at the algorithmic level.

The performances of the optimized implementation for all instances is listed in Table 6.

Table 6. Performances of the optimized implementation of TRIKE (in kiloCycles)

Instance	TRIKE-1	TRIKE-2	TRIKE-3	TRIKE-5	TRIKE-7	TRIKE-9
KeyGen	3,122	5,058	8,627	15,260	39,550	65,422
Encaps	1,682	2,268	3,025	4,735	9,221	12,493
Decaps	2,060	3,062	4,318	7,768	19,048	36,187

It should be noted that the performance figures reported in Table 6 are significantly influenced by the hash and PRG interfaces provided by the ICCS benchmarking framework. Since these interfaces differ from those commonly used in the NIST PQC software benchmarks, the resulting cycle counts are not directly comparable with the published results of other code-based KEMs.

To enable a fair comparison with NIST candidates, we replaced the ICCS hash/PRG interfaces with the same AES and SHA2 implementations adopted in the BIKE optimized implementation. Under this setting, we again benchmarked the optimized implementation of TRIKE-1, TRIKE-3 and TRIKE-5. The results are compared with the performance of BIKE as summarized in Table 7.

Table 7. Performance comparison between optimized TRIKE and BIKE under the same implementation interface (in kiloCycles)

	Level 1		Level 3		Level 5	
Instance	TRIKE-1	BIKE-1	TRIKE-3	BIKE-3	TRIKE-5	BIKE-5
KeyGen	2,485	366	7,683	1,049	13,947	2,643
Encaps	213	74	482	164	1,110	325
Decaps	596	1,177	1,761	3,512	3,999	9,106

Due to the additional arithmetical operations required by TRIKE, neither KeyGen nor Encaps is able to outperform BIKE in terms of execution time. Nevertheless, TRIKE benefits from a significantly smaller parameter d , which directly reduces the complexity of the bit-flipping decoding process and leads to substantial gains in decapsulation performance. Specifically, at NIST Security Levels 1, 3, and 5, the Decaps execution time of TRIKE is reduced to 51%, 50%, and 44% of that of BIKE, respectively. The combined encapsulation and decapsulation time is reduced to 65%, 61%, and 54% of BIKE's corresponding cost at Security Levels 1, 3, and 5, respectively. Furthermore, this performance trade-off is highly advantageous in practical deployments. In many real-world scenarios, such as high-traffic servers handling numerous incoming TLS connections, decapsulation is executed far more frequently than key generation or encapsulation. Consequently, TRIKE's significantly faster decapsulation provides a crucial operational benefit, improving overall system throughput and reducing server-side latency where it matters most.

3 Decryption Failure Rate Analysis

3.1 Theoretical DFR Model

The analysis of the DFR of the Normalized Min-Sum decoder follows the finite-length scaling behavior of iterative decoding in [11]. For codes under iterative decoding, the DFR of waterfall region admits the following approximation

$$\text{DFR} = Q\left(\frac{\sqrt{n}(C(\sigma) - C(\sigma^*))}{\alpha}\right),$$

where Q is the Gaussian Q-function, σ denotes the channel parameter, and $C(\sigma)$ is the corresponding channel capacity. The parameter σ^* represents the decoding threshold, while α is a scaling parameter that characterizes the slope of the waterfall region and depends on both the code and the decoding algorithm.

For the QC-MDPC code, the effective channel capacity is given by

$$C(\sigma) = 1 - \frac{t}{n}.$$

Although this expression provides a theoretical characterization of the DFR, directly computing the parameters $C(\sigma^*)$ and α for a specific decoder with limited precision and fixed number of iterations is highly non-trivial. Therefore, instead of relying on a purely theoretical evaluation, we adopt a practical approach by estimating these parameters via the finite-length scaling law based on empirical DFR measurements.

Applying the inverse Q -function, we obtain

$$\frac{Q^{-1}(\text{DFR})}{\sqrt{n}} = -\frac{t}{\alpha} \cdot \frac{1}{n} + \frac{1 - C(\sigma^*)}{\alpha}.$$

Define $y = Q^{-1}(\text{DFR})/\sqrt{n}$, $x = 1/n$. Then y and x exhibit a linear relationship

$$y = ax + b,$$

where $a = -t/\alpha$ and $b = (1 - C(\sigma^*))/\alpha$.

This linear form enables ordinary least-squares fitting for parameter estimation and serves as the basis of the DFR extrapolation method used below.

3.2 Validation of the FLS-Based DFR Model

To validate the suitability of the proposed FLS-based linear model, we compare its fitting performance with the conventional extrapolation method commonly used in QC-MDPC-based cryptosystems from [6]. In that approach, the curve in the $(r, \log_2(\text{DFR}))$ domain is typically concave over the parameter range of interest, and a conservative upper bound is obtained by extrapolating along the secant determined by the last two available data points.

Using a representative dataset measured for the TRIKE-1 instance, the FLS linear model exhibits a significantly better fitting accuracy. In particular, the coefficient of determination reaches $R^2 = 0.999987$, compared to $R^2 = 0.972303$ for the direct linear fit in the $(r, \log_2(\text{DFR}))$ domain.

This difference is consistent with the observed geometric behavior of the two models. The function $r \mapsto \log_2(\text{DFR})$ exhibits a concave profile with decreasing slope, so a global linear approximation is inherently inaccurate. For this reason, previous estimates typically rely only on the secant determined by the last two points to obtain a conservative upper bound, as illustrated in Figure 1. However, such a construction does not exploit all available data and generally produces a looser estimate.

By contrast, after the FLS transformation, the relation between

$$y = \frac{Q^{-1}(\text{DFR})}{\sqrt{n}} \quad \text{and} \quad x = \frac{1}{n}$$

is observed to be highly linear, as shown in Figure 2, making ordinary least-squares fitting directly applicable. The resulting regression captures all measured points with substantially smaller residual error and provides a tighter extrapolation toward the target parameter.

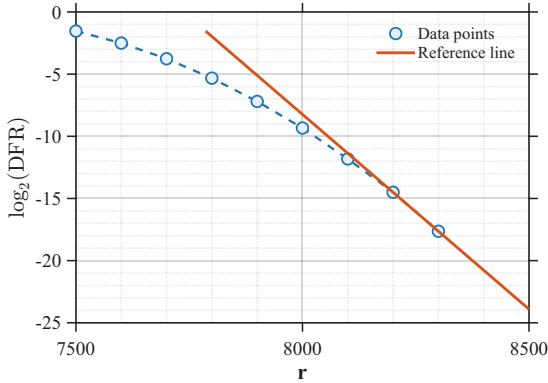


Fig. 1. Conventional linear extrapolation.

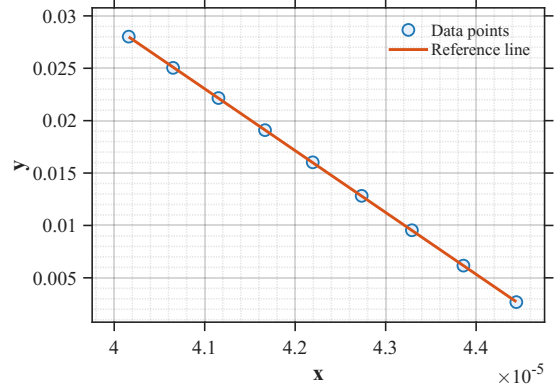


Fig. 2. FLS-based least-squares fitting.

This comparison suggests that the FLS-based method yields a more accurate upper bound on the DFR than conventional concavity-based extrapolation, while retaining a principled analytical justification. We therefore adopt the FLS model in the concrete DFR estimation.

3.3 Concrete DFR Estimate

To estimate the DFR at our target parameter r_{test} , we performed independent decoding experiments for relatively small parameters r . Let F denote the number of failures in N experiments. Then

$$F \sim \text{Bin}(N, \theta),$$

where $\theta = \text{DFR}$ is the true failure probability.

Using the Clopper-Pearson interval, a $(1 - \alpha)$ -confidence interval for θ is given by

$$\theta_- = B^{-1}(\alpha/2, F, N - F + 1), \quad \theta_+ = B^{-1}(1 - \alpha/2, F + 1, N - F),$$

where B^{-1} is the inverse of $x \mapsto B(x, a, b)$, the incomplete beta function:

$$B(x, a, b) = \frac{\Gamma(a+b)}{\Gamma(a)\Gamma(b)} \int_0^x t^{a-1}(1-t)^{b-1} dt.$$

This yields a confidence interval for DFR, which can be transformed into a confidence interval for

$$y = \frac{Q^{-1}(\text{DFR})}{\sqrt{n}}.$$

When multiple confidence intervals are calculated, for any pair (r_i, r_j) such that $r_i < r_j < r_{test}$ with their corresponding x_i, x_j and confidence intervals for y_i, y_j , the result of y_{test} is decided by linear interpolation

$$y_{test} = (1+A)y_j - Ay_i,$$

where

$$A = \frac{x_j - x_{test}}{x_i - x_j} > 0.$$

A conservative confidence interval for y_{test} is derived by combining the extrema of the confidence intervals of y_i and y_j . Specifically, the upper (lower) bound of y_{test} is obtained by using the lower (upper) bound of y_i together with the upper (lower) bound of y_j

$$y_{test}^- = (1+A)y_j^- - Ay_i^+, \quad y_{test}^+ = (1+A)y_j^+ - Ay_i^-.$$

Repeating this procedure for all pairs (i, j) yields multiple candidate intervals. Among all such candidate intervals, we select the one with the smallest upper bound as the final estimate, which gives the tightest upper bound on y_{test} . Finally, the DFR at r_{test} is recovered via

$$\text{DFR}_{test} = Q(y_{test}^+ \cdot \sqrt{n_{test}}).$$

By fixing bit-flipping decoding to 7 iterations, we evaluate the DFR of our scheme under small-scale parameters. The theoretical estimates calculated via our model are presented in Table 8.

Table 8. DFR with a 95% confidence level

	TRIKE-1	TRIKE-2	TRIKE-3	TRIKE-5	TRIKE-7	TRIKE-9
r	10301	15541	22003	35363	69691	114043
$\log_2(\text{DFR})$	-128.33	-128.41	-195.64	-256.44	-431.23	-525.71

3.4 Weak Key Rejection

The FLS-based DFR model provides an accurate estimate of the decoding failure rate of the TRIKE bit-flipping decoder for the selected parameter sets. However, it characterizes the typical behaviour of the decoder and may not fully capture failures caused by specific structural properties of the key or error patterns. In particular, two types of phenomena may affect the DFR: weak keys, namely decoding failures induced by structured keys, and error floors, namely decoding failures induced by structured error patterns. While our simulations show good agreement with the FLS-based predictions, it remains difficult to precisely quantify the contribution of the error floor in the very low-DFR regime. In fact, in coding theory, accurately characterizing the low-DFR regime has long been a challenging open problem.

To efficiently and in a unified manner address the weak key classes identified in [6,12], we employ a weak-key rejection procedure inspired by the methodology of [13]. Specifically, generated keys are filtered according to the number of 4-cycles both within individual circulant blocks and across different circulant blocks. Keys exceeding the prescribed thresholds are discarded and regenerated. Further details of the weak key rejection procedure are provided in Appendix A.

4 Security Proofs

4.1 Security Assumptions and Hardness Reduction

In this subsection, we present the formal definitions of the underlying security assumptions. We then show that the hardness of these assumptions can be reduced to well-established problems, such as 3-QCCF and (2,3)-QCSD.

Specifically, we first introduce the Ratio Quasi-Cyclic Codeword Finding (RQCCF) problem and prove that its hardness reduces to that of the standard 3-QCCF problem.

Definition 5 (3-QCCF Distribution). *The 3-QCCF distribution samples vectors $h_0, h_1, h_2 \xleftarrow{\$} K(w)$, $x \xleftarrow{\$} \mathcal{R}_{\text{even}}$, computes $y \leftarrow \frac{h_0 + xh_1}{h_2}$ and outputs (x, y) .*

Definition 6 (3-QCCF Problem). *The 3-QCCF problem asks to decide with non-negligible advantage whether (x, y) came from 3-QCCF distribution or the uniform distribution $\mathcal{R}_{\text{even}} \times \mathcal{R}_{\text{odd}}$.*

Definition 7 (RQCCF Distribution). *The RQCCF distribution samples vectors $h_0, h_1, h_2 \xleftarrow{\$} K(w)$, $t_1, t_2 \xleftarrow{\$} \mathcal{R}_{\text{even}} \times \mathcal{R}_{\text{even}}$, computes $x \leftarrow \frac{h_2 + t_2 h_0}{h_1 + t_1 h_0}$ and outputs (t_1, t_2, x) .*

Definition 8 (RQCCF Problem). *The RQCCF problem asks to decide with non-negligible advantage whether (t_1, t_2, x) came from RQCCF distribution or the uniform distribution $\mathcal{R}_{\text{even}} \times \mathcal{R}_{\text{even}} \times \mathcal{R}_{\text{odd}}$.*

Lemma 1 (Bijection Preserves Uniform Independence). *Let $S_1, \dots, S_m$ and $T_1, \dots, T_m$ be finite sets, and let*

$$f : S_1 \times \dots \times S_m \rightarrow T_1 \times \dots \times T_m$$

be a bijection. If $(x_1, \dots, x_m)$ is uniformly distributed over $S_1 \times \dots \times S_m$, then

$$(y_1, \dots, y_m) = f(x_1, \dots, x_m)$$

is uniformly distributed over $T_1 \times \dots \times T_m$. In particular, $y_1, \dots, y_m$ are mutually independent and uniformly distributed over $T_1, \dots, T_m$, respectively.

Proof. Let $N = \prod_{i=1}^m |S_i| = \prod_{i=1}^m |T_i|$, where the equality follows from the bijectivity of f .

Since $(x_1, \dots, x_m)$ is uniformly distributed over $S_1 \times \dots \times S_m$, for any $(s_1, \dots, s_m)$,

$$\Pr[(x_1, \dots, x_m) = (s_1, \dots, s_m)] = \frac{1}{N}.$$

Let $(y_1, \dots, y_m) = f(x_1, \dots, x_m)$. For any $(t_1, \dots, t_m)$, by bijectivity there exists a unique $(s_1, \dots, s_m)$ such that

$$f(s_1, \dots, s_m) = (t_1, \dots, t_m).$$

Hence

$$\Pr[(y_1, \dots, y_m) = (t_1, \dots, t_m)] = \frac{1}{N},$$

which shows that $(y_1, \dots, y_m)$ is uniformly distributed over $T_1 \times \dots \times T_m$.

For any $i \in \{1, \dots, m\}$ and $t_i \in T_i$, we have

$$\Pr[y_i = t_i] = \sum_{\substack{t_j \in T_j \\ j \neq i}} \Pr[(y_1, \dots, y_m) = (t_1, \dots, t_m)] = \frac{\prod_{j \neq i} |T_j|}{N} = \frac{1}{|T_i|}.$$

Finally, for any $(t_1, \dots, t_m)$,

$$\Pr[y_1 = t_1, \dots, y_m = t_m] = \frac{1}{N} = \prod_{i=1}^m \frac{1}{|T_i|} = \prod_{i=1}^m \Pr[y_i = t_i],$$

which proves mutual independence.

Theorem 1 *If there exists a PPT adversary that solves the RQCCF problem with advantage ϵ , then there exists a PPT adversary that solves the 3-QCCF problem with the same advantage ϵ .*

Proof. Adversary Construction. Let $\mathcal{A}$ be an adversary for the RQCCF problem. We construct an adversary $\mathcal{B}$ for the 3-QCCF problem as follows.

- Given a challenge (x, y) , $\mathcal{B}$ samples $t_1 \xleftarrow{\$} \mathcal{R}_{\text{even}}$ and sets $t_2 \leftarrow x + t_1 y$.
- $\mathcal{B}$ runs $\mathcal{A}$ with input (t_1, t_2, y) and receives the output of $\mathcal{A}$.
- If $\mathcal{A}$ answers the 3-QCCF distribution, then $\mathcal{B}$ outputs the RQCCF distribution. Otherwise, $\mathcal{B}$ outputs the uniform distribution.

Distribution matching. We then analyze the two cases.

(1) (x, y) is sampled from the 3-QCCF distribution. Then there exist $h_0, h_1, h_2 \xleftarrow{\$} K(w)$ such that $y = \frac{h_0 + xh_1}{h_2}$. We first show that t_2 is uniformly distributed over $\mathcal{R}_{\text{even}}$ and independent of t_1 . Substituting y , we rewrite t_2 as

$$t_2 = x + t_1 y = x + t_1 \cdot \frac{h_0 + xh_1}{h_2} = x \left(1 + \frac{t_1 h_1}{h_2} \right) + \frac{t_1 h_0}{h_2}.$$

Let

$$A = 1 + \frac{t_1 h_1}{h_2}, \quad B = \frac{t_1 h_0}{h_2}.$$

With $t_1 \in \mathcal{R}_{\text{even}}$ and $1 \in \mathcal{R}_{\text{odd}}$, we have $A \in \mathcal{R}_{\text{odd}}$. Since every element in $\mathcal{R}_{\text{odd}}$ is invertible in R , the map

$$(t_1, x) \mapsto (t_1, t_2 = Ax + B)$$

is a bijection over $\mathcal{R}_{\text{even}}$. With the bijection from (t_1, x) to (t_1, t_2) and Lemma 1, (t_1, t_2) is uniform random on $\mathcal{R}_{\text{even}} \times \mathcal{R}_{\text{even}}$.

Next, we show that (t_1, t_2, y) follows the RQCCF distribution. By definition, it suffices to show that there exist $\bar{h}_0, \bar{h}_1, \bar{h}_2 \xleftarrow{\$} K(w)$ such that

$$y = \frac{\bar{h}_2 + t_2 \bar{h}_0}{\bar{h}_1 + t_1 \bar{h}_0}.$$

Define

$$\bar{h}_0 := h_1, \quad \bar{h}_1 := h_2, \quad \bar{h}_2 := h_0.$$

Then the above equality becomes

$$y = \frac{h_0 + t_2 h_1}{h_2 + t_1 h_1}.$$

To prove this equality, it suffices to verify that

$$y(h_2 + t_1 h_1) = h_0 + t_2 h_1.$$

Substituting $t_2 = x + t_1 y$, the right-hand side becomes

$$h_0 + (x + t_1 y)h_1 = h_0 + xh_1 + t_1 y h_1.$$

Thus, it suffices to show that

$$yh_2 + t_1 y h_1 = h_0 + xh_1 + t_1 y h_1.$$

Cancelling the common term $t_1 y h_1$ from both sides, we reduce to

$$yh_2 = h_0 + xh_1,$$

which holds by the definition of y in the 3-QCCF distribution.

Therefore,

$$y = \frac{\bar{h}_2 + t_2 \bar{h}_0}{\bar{h}_1 + t_1 \bar{h}_0},$$

and hence (t_1, t_2, y) is distributed exactly as an RQCCF sample.

(2) (x, y) is uniformly sampled from $\mathcal{R}_{\text{even}} \times \mathcal{R}_{\text{odd}}$. With $t_2 = x + t_1 y$, we can construct a natural bijection from (t_1, x, y) to (t_1, t_2, y) . With Lemma 1, (t_1, t_2, y) is uniformly distributed over $\mathcal{R}_{\text{even}} \times \mathcal{R}_{\text{even}} \times \mathcal{R}_{\text{odd}}$.

Conclusion. In both cases, the distribution given to $\mathcal{A}$ is identical to its respective distribution of RQCCF problem. Therefore, $\mathcal{B}$ perfectly simulates the RQCCF problem, and its advantage is equal to that of $\mathcal{A}$, namely ϵ .

Then we introduce the definition of 3-TRIKE problem, which corresponds to the security of key recovery attack of our scheme. Furthermore, we prove that the hardness of 3-TRIKE can be reduced to that of RQCCF.

Definition 9 (3-TRIKE Distribution). *The 3-TRIKE distribution samples vectors $h_0, h_1, h_2 \xleftarrow{\$} K(w)$, $t_1, t_2, r_1 \xleftarrow{\$} \mathcal{R}_{\text{even}} \times \mathcal{R}_{\text{even}} \times \mathcal{R}_{\text{odd}}$, computes $r_2 \leftarrow t_2 + (r_1 + t_1) \frac{h_2 + t_2 h_0}{h_1 + t_1 h_0}$ and outputs (t_1, t_2, r_1, r_2) .*

Definition 10 (3-TRIKE Problem). *The 3-TRIKE problem asks to decide with non-negligible advantage whether (t_1, t_2, r_1, r_2) came from 3-TRIKE distribution or the uniform distribution $\mathcal{R}_{\text{even}} \times \mathcal{R}_{\text{even}} \times \mathcal{R}_{\text{odd}} \times \mathcal{R}_{\text{odd}}$.*

Theorem 2 *If there exists a PPT adversary that solves the 3-TRIKE problem with advantage ϵ , then there exists a PPT adversary that solves the RQCCF problem with advantage ϵ .*

Proof. Adversary Construction. Let $\mathcal{A}$ be an adversary for 3-TRIKE problem. Then we construct an adversary $\mathcal{B}$ for the RQCCF problem as follows.

- Given an instance (t_1, t_2, x) of the RQCCF problem, $\mathcal{B}$ samples $r_1 \xleftarrow{\$} \mathcal{R}_{\text{odd}}$ and computes $r_2 := t_2 + (r_1 + t_1)x$.
- $\mathcal{B}$ runs $\mathcal{A}$ with input (t_1, t_2, r_1, r_2) and receives the output of $\mathcal{A}$.
- If $\mathcal{A}$ outputs that the input is from the 3-TRIKE distribution, $\mathcal{B}$ outputs the RQCCF distribution. Otherwise, $\mathcal{B}$ outputs the uniform distribution.

Distribution matching. We also analyze two cases.

(1) $x = \frac{h_2 + t_2 h_0}{h_1 + t_1 h_0}$ as in the RQCCF distribution. Then (t_1, t_2, r_1, r_2) is distributed exactly as 3-TRIKE with the definition of 3-TRIKE.

(2) x is uniform in $\mathcal{R}_{\text{odd}}$. Then we can construct a bijection from (t_1, t_2, r_1, x) to (t_1, t_2, r_1, r_2) . With Lemma 1, (t_1, t_2, r_1, r_2) is uniform random distribution on $\mathcal{R}_{\text{even}} \times \mathcal{R}_{\text{even}} \times \mathcal{R}_{\text{odd}} \times \mathcal{R}_{\text{odd}}$.

Conclusion. Therefore, the advantage of $\mathcal{B}$ in distinguishing RQCCF from uniform is exactly the same as the advantage of $\mathcal{A}$ in distinguishing TRIKE distribution from uniform.

Next, we introduce the (2,3)-TRIKE problem, which characterizes the security of message recovery attacks, and prove that its hardness reduces to that of (2,3)-QCSD.

Definition 11 ((2,3)-QCSD Distribution). *The (2,3)-QCSD distribution samples vectors $e_0, e_1, e_2 \xleftarrow{\$} E(t)$, $x, y \xleftarrow{\$} \mathcal{R}_{\text{even}} \times \mathcal{R}_{\text{odd}}$, computes $u \leftarrow e_0 + x e_2$, $v \leftarrow e_1 + y e_2$ and outputs (x, y, u, v) .*

For any integer t , its parity is denoted $p(t) \in \{\text{odd}, \text{even}\}$. Let $\mathcal{R}_{2,p(t)} = (\mathcal{R}_{\text{even}} \times \mathcal{R}_{p(t)}) \cup (\mathcal{R}_{\text{odd}} \times \mathcal{R}_{p(t+1)})$.

Definition 12 ((2,3)-QCSD Problem). *The (2,3)-QCSD problem asks to decide with non-negligible advantage whether (x, y, u, v) came from (2,3)-QCSD distribution or the uniform distribution $\mathcal{R}_{\text{even}} \times \mathcal{R}_{\text{odd}} \times \mathcal{R}_{2,p(t)}$.*

Definition 13 ((2,3)-TRIKE Distribution). *The (2,3)-TRIKE distribution samples vectors $e_0, e_1, e_2 \xleftarrow{\$} E(t)$, $t_1, t_2, r_1, r_2 \xleftarrow{\$} \mathcal{R}_{\text{even}} \times \mathcal{R}_{\text{even}} \times \mathcal{R}_{\text{odd}} \times \mathcal{R}_{\text{odd}}$, computes $u \leftarrow e_0 + r_1 e_1 + r_2 e_2$, $v \leftarrow e_0 + t_1 e_1 + t_2 e_2$ and outputs $(t_1, t_2, r_1, r_2, u, v)$.*

Definition 14 ((2,3)-TRIKE Problem). The (2,3)-TRIKE problem asks to decide with non-negligible advantage whether $(t_1, t_2, r_1, r_2, u, v)$ came from (2,3)-TRIKE distribution or the uniform distribution $\mathcal{R}_{\text{even}} \times \mathcal{R}_{\text{even}} \times \mathcal{R}_{\text{odd}} \times \mathcal{R}_{\text{odd}} \times \mathcal{R}_{p(t)} \times \mathcal{R}$.

Theorem 3 If there exists a PPT adversary that solves the (2,3)-TRIKE problem with advantage ϵ , then there exists a PPT adversary that solves the (2,3)-QCSD problem with advantage ϵ .

Proof. Adversary Construction. Let $\mathcal{A}$ be an adversary for solving (2,3)-TRIKE problem. Then we construct an adversary $\mathcal{B}$ for the (2,3)-QCSD problem as follows.

- Given an instance (x, y, u, v) of (2,3)-QCSD problem, $\mathcal{B}$ samples $t_1, r_1 \xleftarrow{\$} \mathcal{R}_{\text{even}} \times \mathcal{R}_{\text{odd}}$.
- $\mathcal{B}$ set $(t_2, r_2, u', v') = (t_1 y + x, r_1 y + x, r_1 v + u, t_1 v + u)$.
- $\mathcal{B}$ runs $\mathcal{A}$ with input $(t_1, t_2, r_1, r_2, u', v')$.
- If $\mathcal{A}$ outputs that the input is from (2,3)-TRIKE distribution, $\mathcal{B}$ outputs the (2,3)-QCSD distribution. Otherwise, $\mathcal{B}$ outputs the uniform distribution.

Distribution Matching. We first prove that the distribution (t_1, r_1, t_2, r_2) is identical to the uniform distribution over $\mathcal{R}_{\text{even}} \times \mathcal{R}_{\text{odd}} \times \mathcal{R}_{\text{even}} \times \mathcal{R}_{\text{odd}}$. Since $t_1 + r_1 \in \mathcal{R}_{\text{odd}}$, the equations $t_2 = t_1 y + x, r_2 = r_1 y + x$ have an unique solution for (x, y) . Thus the map from (t_1, r_1, x, y) to (t_1, r_1, t_2, r_2) is a bijection. With Lemma 1, (t_1, r_1, t_2, r_2) is a uniform distribution over $\mathcal{R}_{\text{odd}} \times \mathcal{R}_{\text{even}} \times \mathcal{R}_{\text{odd}}$.

Then we analyze two cases.

(1) (x, y, u, v) follows (2,3)-QCSD distribution. So $\exists e_0, e_1, e_2 \xleftarrow{\$} E(t)$ s.t. $u = e_0 + x e_2$ and $v = e_1 + y e_2$. With $u' = r_1 v + u$ and $v' = t_1 v + u$, we can verify that $u' = r_1 v + u = r_1 e_1 + r_1 y e_2 + e_0 + x e_2 = e_0 + r_1 e_1 + r_2 e_2$ and $v' = t_1 v + u = t_1 e_1 + t_1 y e_2 + e_0 + x e_2 = e_0 + t_1 e_1 + t_2 e_2$, which is exactly the definition of (2,3)-TRIKE distribution.

(2) (x, y, u, v) follows uniform distribution. Since $t_1 + r_1 \in \mathcal{R}_{\text{odd}}$, the equations $u' = r_1 v + u, v' = t_1 v + u$ have an unique solution for (u, v) . Thus the map from $(t_1, r_1, t_2, r_2, u, v)$ to $(t_1, r_1, t_2, r_2, u', v')$ is a bijection. With Lemma 1, (t_1, r_1, u', v') is a uniform distribution over $\mathcal{R}_{\text{even}} \times \mathcal{R}_{\text{odd}} \times \mathcal{R}_{\text{even}} \times \mathcal{R}_{\text{odd}} \times \mathcal{R}_{p(t)} \times \mathcal{R}$.

Conclusion. As a result, $\mathcal{B}$ succeeds if and only if the $\mathcal{A}$ solves the (2,3)-TRIKE problem, which completes the proof.

4.2 OW-CPA Security

We first restate the public-key encryption scheme PKE_0 :

- KeyGen_0 :
 - Output: $(h_0, h_1, h_2) \in K(w)$, $r_1 \in \mathcal{R}_{\text{odd}}$, $r_2 \in \mathcal{R}_{\text{odd}}$, $t_0 \in \mathcal{R}_{\text{even}}$, $t_1 \in \mathcal{R}_{\text{even}}$, $t_2 \in \mathcal{R}_{\text{even}}$
 - $h_0, h_1, h_2 \xleftarrow{\$} K(w)$
 - $t_1, t_2, r_1 \xleftarrow{\$} \mathcal{R}_{\text{even}} \times \mathcal{R}_{\text{even}} \times \mathcal{R}_{\text{odd}}$
 - $t_0 \leftarrow \frac{r_1 h_0 + h_1}{r_1 + t_1}, r_2 \leftarrow \frac{h_2 + t_0 t_2}{t_0 + h_0}$
- Encrypt_0 :
 - Input: $r_1 \in \mathcal{R}_{\text{odd}}$, $r_2 \in \mathcal{R}_{\text{odd}}$, $t_1 \in \mathcal{R}_{\text{even}}$, $t_2 \in \mathcal{R}_{\text{even}}$, $(e_0, e_1, e_2) \in E(t)$
 - Output: $u \in \mathcal{R}_{p(t)}$, $v \in \mathcal{R}$
 - $u \leftarrow e_0 + e_1 r_1 + e_2 r_2, v \leftarrow e_0 + e_1 t_1 + e_2 t_2$
- Decrypt_0 :
 - Input: $(h_0, h_1, h_2) \in K(w)$, $r_1 \in \mathcal{R}_{\text{odd}}$, $r_2 \in \mathcal{R}_{\text{odd}}$, $t_0 \in \mathcal{R}_{\text{even}}$, $t_1 \in \mathcal{R}_{\text{even}}$, $t_2 \in \mathcal{R}_{\text{even}}$, $u \in \mathcal{R}_{p(t)}$, $v \in \mathcal{R}$
 - Output: $(e'_0, e'_1, e'_2) \in E(t) \cup \{\perp\}$
 - $s \leftarrow (h_0 + t_0)u - t_0 v$
 - $(e'_0, e'_1, e'_2) \leftarrow \text{Decoder}(h_0, h_1, h_2, s)$

We prove that the public-key encryption scheme PKE_0 is OW-CPA secure under the hardness assumptions of the 3-QCCF and (2,3)-QCSD problems.

We denote by $\mathcal{A}'$ a PPT OW-CPA adversary against PKE_0 . The proof proceeds via a sequence of games.

Game 1 $G_1(\mathcal{A}')$
1. $h_0, h_1, h_2 \xleftarrow{\\$} K(w)$ 2. $t_1, t_2, r_1 \xleftarrow{\\$} \mathcal{R}_{\text{even}} \times \mathcal{R}_{\text{even}} \times \mathcal{R}_{\text{odd}}$ 3. $r_2 \leftarrow t_2 + (r_1 + t_1) \frac{h_2 + t_2 h_0}{h_1 + t_1 h_0}$ 4. $(e_0^*, e_1^*, e_2^*) \xleftarrow{\\$} E(t)$ 5. $u \leftarrow e_0^* + e_1^* r_1 + e_2^* r_2, v \leftarrow e_0^* + e_1^* t_1 + e_2^* t_2$ 6. $e \leftarrow \mathcal{A}'(t_1, t_2, r_1, r_2, u, v)$ 7. return $(e = e^*)$

G_1 is the OW-CPA game of PKE_0 . By definition,

$$\text{Adv}_{\text{PKE}_0}^{\text{OW-CPA}}(\mathcal{A}') = \text{Adv}_{G_1}(\mathcal{A}').$$

Game 2 $G_2(\mathcal{A}')$
1. $h_0, h_1, h_2 \xleftarrow{\\$} K(w)$ 2. $t_1, t_2, r_1 \xleftarrow{\\$} \mathcal{R}_{\text{even}} \times \mathcal{R}_{\text{even}} \times \mathcal{R}_{\text{odd}}$ 3. $r_2 \xleftarrow{\\$} \mathcal{R}_{\text{odd}}$ 4. $(e_0^*, e_1^*, e_2^*) \xleftarrow{\\$} E(t)$ 5. $u \leftarrow e_0^* + e_1^* r_1 + e_2^* r_2, v \leftarrow e_0^* + e_1^* t_1 + e_2^* t_2$ 6. $e \leftarrow \mathcal{A}'(t_1, t_2, r_1, r_2, u, v)$ 7. return $(e = e^*)$

In G_2 , r_2 is replaced by a uniformly random element sampled from $\mathcal{R}_{\text{odd}}$. Suppose there exists a PPT adversary $\mathcal{A}'_1$ that distinguishes G_1 from G_2 with advantage ϵ . Then, there exists a PPT adversary $\mathcal{A}_1$ that solves the 3-TRIKE problem with the same advantage ϵ :

- Given an instance (t_1, t_2, r_1, r_2) of the 3-TRIKE problem.
- $\mathcal{A}_1$ samples $(e_0^*, e_1^*, e_2^*) \xleftarrow{\$} E(t)$.
- $\mathcal{A}_1$ computes $u \leftarrow e_0^* + e_1^* r_1 + e_2^* r_2, v \leftarrow e_0^* + e_1^* t_1 + e_2^* t_2$.
- $\mathcal{A}_1$ runs $\mathcal{A}'_1(t_1, t_2, r_1, r_2, u, v)$ to obtain the type of game.
- If $\mathcal{A}'_1$ outputs G_1 , $\mathcal{A}_1$ outputs (t_1, t_2, r_1, r_2) came from 3-TRIKE distribution.
- If $\mathcal{A}'_1$ outputs G_2 , $\mathcal{A}_1$ outputs (t_1, t_2, r_1, r_2) came from the uniform distribution.

Hence,

$$|\text{Adv}_{G_1}(\mathcal{A}') - \text{Adv}_{G_2}(\mathcal{A}')| \leq \text{Adv}_{3\text{-TRIKE}}(\mathcal{A}_1) \leq \text{Adv}_{3\text{-QCCF}}(\mathcal{B}_1).$$

Game 3 $G_3(\mathcal{A}')$
1. $h_0, h_1, h_2 \xleftarrow{\\$} K(w)$ 2. $t_1, t_2, r_1 \xleftarrow{\\$} \mathcal{R}_{\text{even}} \times \mathcal{R}_{\text{even}} \times \mathcal{R}_{\text{odd}}$ 3. $r_2 \xleftarrow{\\$} \mathcal{R}_{\text{odd}}$ 4. $(e_0^*, e_1^*, e_2^*) \xleftarrow{\\$} E(t)$ 5. $u, v \xleftarrow{\\$} \mathcal{R}_{p(t)} \times \mathcal{R}$ 6. $e \leftarrow \mathcal{A}'(t_1, t_2, r_1, r_2, u, v)$ 7. return $(e = e^*)$

G_3 is obtained from G_2 by replacing (u, v) with uniformly random elements. Suppose there exists a PPT adversary $\mathcal{A}'_2$ that distinguishes G_2 from G_3 with advantage ϵ . Then, there exists a PPT adversary $\mathcal{A}_2$ that solves the (2,3)-TRIKE problem with the same advantage ϵ :

- Given an instance $(t_1, t_2, r_1, r_2, u, v)$ of the (2,3)-TRIKE problem.
- $\mathcal{A}_2$ runs $\mathcal{A}'_2(t_1, t_2, r_1, r_2, u, v)$ to obtain the type of game.
- If $\mathcal{A}'_2$ outputs G_2 , $\mathcal{A}_2$ outputs $(t_1, t_2, r_1, r_2, u, v)$ came from (2,3)-TRIKE distribution.
- If $\mathcal{A}'_2$ outputs G_3 , $\mathcal{A}_2$ outputs $(t_1, t_2, r_1, r_2, u, v)$ came from the uniform distribution.

Therefore,

$$|\text{Adv}_{G_2}(\mathcal{A}') - \text{Adv}_{G_3}(\mathcal{A}')| \leq \text{Adv}_{(2,3)\text{-TRIKE}}(\mathcal{A}_2) \leq \text{Adv}_{(2,3)\text{-QCSD}}(\mathcal{B}_2).$$

In G_3 , the variables t_1, t_2, r_1, r_2, u, v are sampled independently of the error vector e^* . Therefore, the adversary $\mathcal{A}'$ receives no information about e^* , and the adversary can do no better than outputting a uniformly random guess:

$$\text{Adv}_{G_3}(\mathcal{A}') = \frac{1}{|\mathcal{E}(t)|}.$$

Theorem 1. *For any PPT OW-CPA adversary $\mathcal{A}'$ against the PKE_0 scheme, there exist PPT distinguishers $\mathcal{B}_1$ and $\mathcal{B}_2$ such that:*

$$\text{Adv}_{\text{PKE}_0}^{\text{OW-CPA}}(\mathcal{A}') \leq \text{Adv}_{3\text{-QCCF}}(\mathcal{B}_1) + \text{Adv}_{(2,3)\text{-QCSD}}(\mathcal{B}_2) + \frac{1}{|\mathcal{E}(t)|}.$$

Proof. Combining the above bounds, we obtain that

$$\begin{aligned} \text{Adv}_{\text{PKE}_0}^{\text{OW-CPA}}(\mathcal{A}') &\leq |\text{Adv}_{G_1}(\mathcal{A}') - \text{Adv}_{G_2}(\mathcal{A}')| + |\text{Adv}_{G_2}(\mathcal{A}') - \text{Adv}_{G_3}(\mathcal{A}')| + \\ &\quad \text{Adv}_{G_3}(\mathcal{A}') \\ &\leq \text{Adv}_{3\text{-QCCF}}(\mathcal{B}_1) + \text{Adv}_{(2,3)\text{-QCSD}}(\mathcal{B}_2) + \frac{1}{|\mathcal{E}(t)|}. \end{aligned}$$

4.3 IND-CPA Security

PKE_1 describes a new encryption scheme which can be viewed as a randomized extension of PKE_0 :

- KeyGen₁:
 - Output: $(h_0, h_1, h_2) \in \mathcal{K}(w)$, $r_1 \in \mathcal{R}_{\text{odd}}$, $r_2 \in \mathcal{R}_{\text{odd}}$, $t_0 \in \mathcal{R}_{\text{even}}$, $t_1 \in \mathcal{R}_{\text{even}}$, $t_2 \in \mathcal{R}_{\text{even}}$
 - $h_0, h_1, h_2 \xleftarrow{\$} \mathcal{K}(w)$
 - $t_1, t_2, r_1 \xleftarrow{\$} \mathcal{R}_{\text{even}} \times \mathcal{R}_{\text{even}} \times \mathcal{R}_{\text{odd}}$
 - $t_0 \leftarrow \frac{r_1 h_0 + h_1}{r_1 + t_1}$, $r_2 \leftarrow \frac{h_2 + t_0 t_2}{t_0 + h_0}$
- Encrypt₁:
 - Input: $r_1 \in \mathcal{R}_{\text{odd}}$, $r_2 \in \mathcal{R}_{\text{odd}}$, $t_1 \in \mathcal{R}_{\text{even}}$, $t_2 \in \mathcal{R}_{\text{even}}$, $m \in \mathcal{M}$
 - Output: $c \in \mathcal{R}_{p(t)} \times \mathcal{R} \times \mathcal{M}$
 - $(e_0, e_1, e_2) \xleftarrow{\$} \mathcal{E}(t)$
 - $u \leftarrow e_0 + e_1 r_1 + e_2 r_2$, $v \leftarrow e_0 + e_1 t_1 + e_2 t_2$
 - $c \leftarrow (u, v, m + \mathcal{L}(e_0, e_1, e_2))$
- Decrypt₁:
 - Input: $(h_0, h_1, h_2) \in \mathcal{K}(w)$, $r_1 \in \mathcal{R}_{\text{odd}}$, $r_2 \in \mathcal{R}_{\text{odd}}$, $t_0 \in \mathcal{R}_{\text{even}}$, $t_1 \in \mathcal{R}_{\text{even}}$, $t_2 \in \mathcal{R}_{\text{even}}$, $(u, v, c_2) \in \mathcal{R}_{p(t)} \times \mathcal{R} \times \mathcal{M}$
 - Output: $m \in \mathcal{M} \cup \{\perp\}$
 - $s \leftarrow (h_0 + t_0)u - t_0 v$
 - $(e'_0, e'_1, e'_2) \leftarrow \text{Decoder}(h_0, h_1, h_2, s)$
 - if $e' = \perp$ then $m \leftarrow \perp$ else $m \leftarrow c_2 + \mathcal{L}(e'_0, e'_1, e'_2)$

The scheme follows a hybrid paradigm, where the algebraic structure induced by the secret-dependent relations implicitly defines a key encapsulation mechanism, while the masking function L , modeled as a random oracle, serves as a one-time pad for the message.

Let $\mathcal{A} = (\mathcal{A}_{choose}, \mathcal{A}_{guess})$ be a PPT IND-CPA adversary against PKE_1 . The adversary is allowed to query the random oracle L , and we denote by L the set of queries made by $\mathcal{A}$ to the random oracle L .

We consider a series of games to prove that PKE_1 is IND-CPA.

Game 4 $G_4(\mathcal{A})$
1. $h_0, h_1, h_2 \xleftarrow{\\$} K(w)$ 2. $t_1, t_2, r_1 \xleftarrow{\\$} \mathcal{R}_{even} \times \mathcal{R}_{even} \times \mathcal{R}_{odd}$ 3. $r_2 \leftarrow t_2 + (r_1 + t_1) \frac{h_2 + t_2 h_0}{h_1 + t_1 h_0}$ 4. $(e_0^*, e_1^*, e_2^*) \xleftarrow{\\$} E(t)$ 5. $u \leftarrow e_0^* + e_1^* r_1 + e_2^* r_2, v \leftarrow e_0^* + e_1^* t_1 + e_2^* t_2$ 6. $(m_0, m_1) \leftarrow \mathcal{A}_{choose}(t_1, t_2, r_1, r_2)$ 7. $b \xleftarrow{\\$} \{0, 1\}$ 8. $c_2 \leftarrow m_b + L(e_0^*, e_1^*, e_2^*)$ 9. $b' \leftarrow \mathcal{A}_{guess}(t_1, t_2, r_1, r_2, u, v, c_2)$ 10. return $(b = b')$

G_4 corresponds exactly to the IND-CPA security experiment of PKE_1 such that

$$\text{Adv}_{\text{PKE}_1}^{\text{IND-CPA}}(\mathcal{A}) = \left| \Pr[G_4(\mathcal{A})] - \frac{1}{2} \right|.$$

To analyze the advantage of the adversary $\mathcal{A}$ based on whether it queries the challenge error e^* , we define the following bad event:

$$\text{BAD} := (e^* \in L).$$

This event occurs if the adversary $\mathcal{A}$ queries the random oracle L with the challenge error vector e^* .

We now define the game G_5 , in which the ciphertext component c_2 is replaced by a uniformly random element in M .

Game 5 $G_5(\mathcal{A})$
1. $h_0, h_1, h_2 \xleftarrow{\\$} K(w)$ 2. $t_1, t_2, r_1 \xleftarrow{\\$} \mathcal{R}_{even} \times \mathcal{R}_{even} \times \mathcal{R}_{odd}$ 3. $r_2 \leftarrow t_2 + (r_1 + t_1) \frac{h_2 + t_2 h_0}{h_1 + t_1 h_0}$ 4. $(e_0^*, e_1^*, e_2^*) \xleftarrow{\\$} E(t)$ 5. $u \leftarrow e_0^* + e_1^* r_1 + e_2^* r_2, v \leftarrow e_0^* + e_1^* t_1 + e_2^* t_2$ 6. $(m_0, m_1) \leftarrow \mathcal{A}_{choose}(t_1, t_2, r_1, r_2)$ 7. $b \xleftarrow{\\$} \{0, 1\}$ 8. $c_2 \xleftarrow{\\$} M$ 9. $b' \leftarrow \mathcal{A}_{guess}(t_1, t_2, r_1, r_2, u, v, c_2)$ 10. return $(b = b')$

We condition on the event BAD not occurring, i.e. $e^* \notin L$. The adversary never queries the random oracle with the challenge error vector e^* . Under the hypothesis that L is a random oracle, the value $L(e^*)$ is indistinguishable from a uniformly random element of M . Hence, $c_2 \leftarrow m_b + L(e_0^*, e_1^*, e_2^*)$ and $c_2 \xleftarrow{\$} M$ follow the same distribution, that is, G_4 and G_5 are indistinguishable conditioned on $\neg \text{BAD}$:

$$\Pr[G_4(\mathcal{A}) \mid \neg \text{BAD}] = \Pr[G_5(\mathcal{A}) \mid \neg \text{BAD}].$$

In G_5 , the ciphertext component c_2 is sampled uniformly at random from M , independently of the challenge bit b . The adversary $\mathcal{A}$ obtains no information about the challenge bit. Thus, conditioned on the event BAD not occurring, we have

$$\Pr[G_5(\mathcal{A}) \mid \neg \text{BAD}] = \frac{1}{2}.$$

To analyze the probability of the bad event BAD , we introduce a modified game based on G_4 . In this game, we keep the execution identical to G_4 , but change the success condition of $\mathcal{A}$ to whether it queries the random oracle L on the challenge error e^* , i.e., whether $e^* \in L$.

Game 6 $G_6(\mathcal{A})$
1. $h_0, h_1, h_2 \xleftarrow{\\$} K(w)$ 2. $t_1, t_2, r_1 \xleftarrow{\\$} \mathcal{R}_{\text{even}} \times \mathcal{R}_{\text{even}} \times \mathcal{R}_{\text{odd}}$ 3. $r_2 \leftarrow t_2 + (r_1 + t_1) \frac{h_2 + t_2 h_0}{h_1 + t_1 h_0}$ 4. $(e_0^*, e_1^*, e_2^*) \xleftarrow{\\$} E(t)$ 5. $u \leftarrow e_0^* + e_1^* r_1 + e_2^* r_2, v \leftarrow e_0^* + e_1^* t_1 + e_2^* t_2$ 6. $(m_0, m_1) \leftarrow \mathcal{A}_{\text{choose}}(t_1, t_2, r_1, r_2)$ 7. $b \xleftarrow{\\$} \{0, 1\}$ 8. $c_2 \leftarrow m_b + L(e_0^*, e_1^*, e_2^*)$ 9. $b' \leftarrow \mathcal{A}_{\text{guess}}(t_1, t_2, r_1, r_2, u, v, c_2)$ 10. return $(e^* \in L)$

G_6 isolates the event that the adversary queries the oracle on the exact error vector $e^* = (e_0^*, e_1^*, e_2^*)$, that is,

$$\Pr[\text{BAD}] = \Pr[G_6(\mathcal{A}_{\text{query}})].$$

We now show that G_6 can be reduced to solving the OW-CPA problem of PKE_0 .

Lemma 2. *For any PPT IND-CPA adversary $\mathcal{A}$ against PKE_1 with queries set L on the random oracle L , there exists a PPT OW-CPA adversary $\mathcal{A}'$ against PKE_0 such that*

$$\Pr[G_6(\mathcal{A})] \leq \text{Adv}_{\text{PKE}_0}^{\text{OW-CPA}}(\mathcal{A}')$$

Proof. Let $\mathcal{A}$ be a PPT IND-CPA adversary against PKE_1 with queries set L on the random oracle L . Then we construct a PPT OW-CPA adversary $\mathcal{A}'$ against PKE_0 as follows.

- Given an instance $(t_1, t_2, r_1, r_2, u, v)$ of the OW-CPA problem of PKE_0 .
- $\mathcal{A}'$ runs $\mathcal{A}$ following the steps of G_6 in $(t_1, t_2, r_1, r_2, u, v)$ to obtain a set $L \subset E(t)$ that $\mathcal{A}$ queries the random oracle L .
- For each $e = (e_0, e_1, e_2) \in L$, checks whether $(e_0 + e_1 r_1 + e_2 r_2 = u) \wedge (e_0 + e_1 t_1 + e_2 t_2 = v)$ is true.
- If such an e exists, output it; otherwise output a random $e \in E(t)$.

By construction, if $\mathcal{A}$ queries the correct e^* , then $\mathcal{A}'$ succeeds in the OW-CPA game. Therefore,

$$\Pr[G_6(\mathcal{A})] \leq \text{Adv}_{\text{PKE}_0}^{\text{OW-CPA}}(\mathcal{A}').$$

Theorem 2. *For any PPT IND-CPA adversary $\mathcal{A}$ against the PKE_1 scheme, there exist PPT adversaries $\mathcal{B}_1$ and $\mathcal{B}_2$ such that:*

$$\text{Adv}_{\text{PKE}_1}^{\text{IND-CPA}}(\mathcal{A}) \leq \frac{1}{2} \left(\text{Adv}_{3\text{-QCCF}}(\mathcal{B}_1) + \text{Adv}_{(2,3)\text{-QCSD}}(\mathcal{B}_2) + \frac{1}{|E(t)|} \right). \quad (2)$$

Proof. Putting all things above together:

$$\begin{aligned}
\Pr[G_4(\mathcal{A})] &= \Pr[G_4(\mathcal{A}) \mid \neg \text{BAD}](1 - \Pr[\text{BAD}]) + \Pr[G_4(\mathcal{A}) \mid \text{BAD}] \Pr[\text{BAD}] \\
&\leq \Pr[G_4(\mathcal{A}) \mid \neg \text{BAD}](1 - \Pr[\text{BAD}]) + \Pr[\text{BAD}] \\
&= \Pr[G_4(\mathcal{A}) \mid \neg \text{BAD}] + (1 - \Pr[G_4(\mathcal{A}) \mid \neg \text{BAD}]) \Pr[\text{BAD}] \\
&\leq \Pr[G_5(\mathcal{A}) \mid \neg \text{BAD}] + (1 - \Pr[G_5(\mathcal{A}) \mid \neg \text{BAD}]) \Pr[G_6(\mathcal{A})] \\
&\leq \frac{1}{2} + \frac{1}{2} \text{Adv}_{\text{PKE}_0}^{\text{OW-CPA}}(\mathcal{A}').
\end{aligned}$$

The advantage of any PPT IND-CPA adversary $\mathcal{A}$ is bounded by

$$\begin{aligned} \text{Adv}_{\text{PKE}_1}^{\text{IND-CPA}}(\mathcal{A}) &= \left| \Pr[G_4(\mathcal{A})] - \frac{1}{2} \right| \\ &\leq \frac{1}{2} \text{Adv}_{\text{PKE}_0}^{\text{OW-CPA}}(\mathcal{A}') \\ &\leq \frac{1}{2} \left(\text{Adv}_{3\text{-QCCF}}(\mathcal{B}_1) + \text{Adv}_{(2,3)\text{-QCSD}}(\mathcal{B}_2) + \frac{1}{|\mathcal{E}(t)|} \right). \end{aligned}$$

4.4 IND-CCA Security

Since our scheme may exhibit decryption failures, we rely on the notion of δ -correctness to quantify the failure probability and incorporate it into the security reduction. A public-key encryption scheme is said to be δ -correct if

$$\mathbb{E} \left[\max_{m \in \mathcal{M}} \Pr[\text{Decrypt}(\text{sk}, c) \neq m \mid c \leftarrow \text{Encrypt}(\text{pk}, m)] \right] \leq \delta,$$

where the expectation is taken over $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}()$.

In our construction, the decryption failure occurs only if the decoding procedure fails. Observe that the variables $(h_0, h_1, h_2, t_1, t_2, r_1, r_2, u, v)$ do not depend on the message m , and the decoding procedure operates solely on these variables. Hence, the failure probability is independent of m , i.e., the scheme is message-agnostic. Therefore, the maximum over m becomes redundant, and the PKE is δ -correct if and only if the DFR of the decoder is less than δ .

Following the HHK framework, we first transform PKE_1 into a deterministic PKE_2 by using random oracles $\mathcal{H}_1, \mathcal{H}_2, \mathcal{H}_3, \mathcal{H}_4$:

- KeyGen_2 :
 - Output: $(h_0, h_1, h_2) \in \mathcal{K}(w)$, $\sigma \in \mathcal{M}$, $r_2 \in \mathcal{R}_{\text{odd}}$
 - $h_0, h_1, h_2 \xleftarrow{\$} \mathcal{K}(w)$
 - $\sigma \xleftarrow{\$} \mathcal{M}$
 - $t_1 \leftarrow \mathcal{H}_1(\sigma)$, $t_2 \leftarrow \mathcal{H}_2(\sigma)$, $r_1 \leftarrow \mathcal{H}_3(\sigma)$
 - $t_0 \leftarrow \frac{r_1 h_0 + h_1}{r_1 + t_1}$, $r_2 \leftarrow \frac{h_2 + t_0 t_2}{t_0 + h_0}$
- Encrypt_2 :
 - Input: $r_2 \in \mathcal{R}_{\text{odd}}$, $\sigma \in \mathcal{M}$, $m \in \mathcal{M}$
 - Output: $c \in \mathcal{R}_{p(t)} \times \mathcal{R} \times \mathcal{M}$
 - $t_1 \leftarrow \mathcal{H}_1(\sigma)$, $t_2 \leftarrow \mathcal{H}_2(\sigma)$, $r_1 \leftarrow \mathcal{H}_3(\sigma)$
 - $(e_0, e_1, e_2) \leftarrow \mathcal{H}_4(m, r_2)$
 - $u \leftarrow e_0 + e_1 r_1 + e_2 r_2$, $v \leftarrow e_0 + e_1 t_1 + e_2 t_2$
 - $c \leftarrow (u, v, m + L(e_0, e_1, e_2))$
- Decrypt_2 :
 - Input: $(h_0, h_1, h_2) \in \mathcal{K}(w)$, $\sigma \in \mathcal{M}$, $c \in \mathcal{R}_{p(t)} \times \mathcal{R} \times \mathcal{M}$
 - Output: $m \in \mathcal{M} \cup \{\perp\}$
 - $t_1 \leftarrow \mathcal{H}_1(\sigma)$, $t_2 \leftarrow \mathcal{H}_2(\sigma)$, $r_1 \leftarrow \mathcal{H}_3(\sigma)$
 - $t_0 \leftarrow \frac{r_1 h_0 + h_1}{r_1 + t_1}$, $r_2 \leftarrow \frac{h_2 + t_0 t_2}{t_0 + h_0}$
 - $m \leftarrow \text{Decrypt}_1((h_0, h_1, h_2), r_1, r_2, t_0, t_1, t_2, c)$
 - if $m \neq \perp$ and $c \neq \text{Encrypt}_2(r_2, \sigma, m)$ then $m \leftarrow \perp$

Under the hypothesis of random oracle, replacing the explicit uniform sampling

$$t_1, t_2, r_1 \xleftarrow{\$} \mathcal{R}_{\text{even}} \times \mathcal{R}_{\text{even}} \times \mathcal{R}_{\text{odd}}$$

with the hash outputs

$$t'_1 = \mathcal{H}_1(\sigma), \quad t'_2 = \mathcal{H}_2(\sigma), \quad r'_1 = \mathcal{H}_3(\sigma)$$

does not affect security because (t_1, t_2, r_1) and (t'_1, t'_2, r'_1) are computationally indistinguishable.

For any PPT OW-PCA adversary $\mathcal{B}'$ of PKE_2 that issues at most q_H queries to $\mathcal{H}_4$ and q_K queries to a plaintext-checking oracle $\mathcal{P}_{\text{CO}}$, by applying the OW-PCA version of [4, Theorem 2], there exists a PPT IND-CPA adversary $\mathcal{A}$ against PKE_1 such that

$$\text{Adv}_{\text{PKE}_2}^{\text{OW-CPA}}(\mathcal{B}') \leq q_H \cdot \delta + \frac{2q_H + 1}{|\mathcal{M}|} + 3 \cdot \text{Adv}_{\text{PKE}_1}^{\text{IND-CPA}}(\mathcal{A}) \quad (3)$$

We now apply the HHK transformation to convert PKE_2 into KEM with implicit rejection:

- KeyGen:
 - Output: $(h_0, h_1, h_2) \in \mathcal{K}(w)$, $\sigma \in \mathcal{M}$, $\sigma' \in \mathcal{M}$, $r_2 \in \mathcal{R}_{\text{odd}}$
 - $h_0, h_1, h_2 \xleftarrow{\$} \mathcal{K}(w)$
 - $\sigma \xleftarrow{\$} \mathcal{M}$, $\sigma' \xleftarrow{\$} \mathcal{M}$
 - $t_1 \leftarrow \mathcal{H}_1(\sigma)$, $t_2 \leftarrow \mathcal{H}_2(\sigma)$, $r_1 \leftarrow \mathcal{H}_3(\sigma)$
 - $t_0 \leftarrow \frac{r_1 h_0 + h_1}{r_1 + t_1}$, $r_2 \leftarrow \frac{h_2 + t_0 t_2}{t_0 + h_0}$
- Encaps:
 - Input: $r_2 \in \mathcal{R}_{\text{odd}}$, $\sigma \in \mathcal{M}$
 - Output: $c \in \mathcal{R}_{p(t)} \times \mathcal{R} \times \mathcal{M}$, $k \in \mathcal{M}$
 - $m \xleftarrow{\$} \mathcal{M}$
 - $c \leftarrow \text{Encrypt}_2(r_2, \sigma, m)$
 - $k \leftarrow \mathcal{K}(m, c)$
- Decaps:
 - Input: $(h_0, h_1, h_2) \in \mathcal{K}(w)$, $\sigma \in \mathcal{M}$, $\sigma' \in \mathcal{M}$, $c \in \mathcal{R}_{p(t)} \times \mathcal{R} \times \mathcal{M}$
 - Output: $k \in \mathcal{M}$
 - $m \leftarrow \text{Decrypt}_2((h_0, h_1, h_2), \sigma, c)$
 - if $m \neq \perp$ then $k \leftarrow \mathcal{K}(m, c)$ else $k \leftarrow \mathcal{K}(\sigma', c)$

It is shown in [4, Theorem 4] that for any PPT IND-CCA adversary $\mathcal{B}$ against KEM making at most q_D queries to the decapsulation oracle $\mathcal{O}_{\text{Decaps}}$ and q_K queries to the random oracle $\mathcal{K}$, there exists a PPT OW-PCA adversary $\mathcal{B}'$ against PKE_2 that issues at most q_K queries to $\mathcal{P}_{\text{CO}}$ such that

$$\text{Adv}_{\text{KEM}}^{\text{IND-CCA}}(\mathcal{B}) \leq \frac{q_K}{|\mathcal{M}|} + \text{Adv}_{\text{PKE}_2}^{\text{OW-PCA}}(\mathcal{B}'). \quad (4)$$

Theorem 3. *For any PPT IND-CCA adversary $\mathcal{B}$ against the KEM making at most q queries to $\mathcal{K}$ or $\mathcal{H}_4$, there exist PPT adversaries $\mathcal{A}$, $\mathcal{B}_1$ and $\mathcal{B}_2$ such that:*

$$\begin{aligned} \text{Adv}_{\text{KEM}}^{\text{IND-CCA}}(\mathcal{B}) &\leq q \cdot \delta + \frac{3q}{|\mathcal{M}|} + 3 \cdot \text{Adv}_{\text{PKE}_1}^{\text{IND-CPA}}(\mathcal{A}) \\ &\leq q \cdot \delta + \frac{3q}{|\mathcal{M}|} + \frac{3}{2} \left(\text{Adv}_{3\text{-QCCF}}(\mathcal{B}_1) + \text{Adv}_{(2,3)\text{-QCSD}}(\mathcal{B}_2) + \frac{1}{|\mathcal{E}(t)|} \right). \end{aligned}$$

Proof. The result follows by combining (3) and (4), and by upper bounding all oracle queries by a single parameter q .

4.5 Security Proof with Weak Key Rejection

We now consider the security implications of introducing weak key rejection into our scheme. Let $\mathcal{K}_{\S}(w) \subseteq \mathcal{K}(w)$ denote the subset of keys that successfully pass the weak key test. The weak key rejection mechanism modifies the key generation phase by replacing the uniform sampling $h_0, h_1, h_2 \xleftarrow{\$} \mathcal{K}(w)$ with $h_0, h_1, h_2 \xleftarrow{\$} \mathcal{K}_{\S}(w)$. We denote this modified scheme as $\text{KEM}_{\S}$.

Theorem 4. *For any PPT IND-CCA adversary $\mathcal{B}$ against $\text{KEM}_{\S}$ where the weak key rejection rate is ρ , there exists a PPT IND-CCA adversary $\mathcal{B}'$ against KEM such that:*

$$\text{Adv}_{\text{KEM}_{\S}}^{\text{IND-CCA}}(\mathcal{B}) = \frac{1}{1 - \rho} \text{Adv}_{\text{KEM}}^{\text{IND-CCA}}(\mathcal{B}').$$

Proof. Let $\mathcal{B}$ be a PPT IND-CCA adversary against $\text{KEM}_\S$. We construct a PPT IND-CCA adversary $\mathcal{B}'$ against the original KEM as follows.

Given an instance of the IND-CCA problem for KEM with public key $\text{pk} = (h_0, h_1, h_2)$, $\mathcal{B}'$ first determines whether the key passes the weak key test, i.e., whether $(h_0, h_1, h_2) \in \text{K}_\S(w)$.

- If $(h_0, h_1, h_2) \in \text{K}_\S(w)$, the given instance perfectly simulates the IND-CCA game for $\text{KEM}_\S$. In this case, $\mathcal{B}'$ forwards the instance to $\mathcal{B}$, answers its oracle queries using its own provided oracles, and outputs the exact response of $\mathcal{B}$. $\mathcal{B}'$ succeeds the IND-CCA game iff $\mathcal{B}$ succeeds the game. This event occurs with probability $1 - \rho$.
- If $(h_0, h_1, h_2) \notin \text{K}_\S(w)$, $\mathcal{B}'$ simply outputs a uniformly random bit $b \xleftarrow{\$} \{0, 1\}$. This event occurs with probability ρ .

Let $\Pr[G_{\text{IND-CCA}}(\mathcal{B})]$ and $\Pr[G_{\text{IND-CCA}}(\mathcal{B}')]$ denote the probabilities that $\mathcal{B}$ and $\mathcal{B}'$ win their respective IND-CCA games. We can express the success probability of $\mathcal{B}'$ as:

$$\Pr[G_{\text{IND-CCA}}(\mathcal{B}')] = (1 - \rho) \Pr[G_{\text{IND-CCA}}(\mathcal{B})] + \rho \cdot \frac{1}{2}.$$

By definition, the advantage of $\mathcal{B}$ against $\text{KEM}_\S$ is:

$$\begin{aligned} \text{Adv}_{\text{KEM}_\S}^{\text{IND-CCA}}(\mathcal{B}) &= \left| \Pr[G_{\text{IND-CCA}}(\mathcal{B})] - \frac{1}{2} \right| \\ &= \left| \frac{1}{1 - \rho} \left(\Pr[G_{\text{IND-CCA}}(\mathcal{B}')] - \frac{\rho}{2} \right) - \frac{1}{2} \right| \\ &= \left| \frac{1}{1 - \rho} \left(\text{Adv}_{\text{KEM}}^{\text{IND-CCA}}(\mathcal{B}') + \frac{1}{2} - \frac{\rho}{2} \right) - \frac{1}{2} \right| \\ &= \frac{1}{1 - \rho} \text{Adv}_{\text{KEM}}^{\text{IND-CCA}}(\mathcal{B}'). \end{aligned}$$

As demonstrated by the theorem, introducing a weak key rejection with rate ρ strictly bounds the advantage amplification by a factor of $\frac{1}{1-\rho}$. Consequently, this incurs a security loss of precisely $-\log_2(1 - \rho)$ bits. As previously discussed in Section 5.1, we have provisioned an adequate security margin within the parameter selection. Therefore, provided ρ remains sufficiently small, the inclusion of weak key rejection does not compromise the overall target security level of the scheme.

5 Security Against Known Attacks

For TRIKE, both key recovery and message recovery attacks can be reduced to decoding problems over quasi-cyclic codes. In particular, key recovery attacks correspond to the 3-TRIKE setting, while message recovery attacks correspond to the (2,3)-TRIKE setting. These problems can be further reduced to the 3-QCCF problem and the (2,3)-QCSD problem, respectively, as shown in Section 4.1.

5.1 Information Set Decoding

The best known attacks against the 3-QCCF problem and the (2,3)-QCSD problem are based on Information Set Decoding (ISD) algorithms. In our analysis, we estimate the attack complexities using the framework of [14], considering several representative ISD variants, including Prange's algorithm [15], Stern's algorithm [16, 17, 18], BJMM [19, 20, 21], Both-May [22] and May-Ozerov [23].

Due to the quasi-cyclic structure of QC-MDPC codes, the overall attack complexities are further reduced by a factor of r for key recovery attacks and by a factor of $\sqrt{r}$ for message recovery attacks. The resulting time complexities at the targeted security levels are summarized in Table 9 and Table 10.

Table 9. Key recovery time complexity (in $\log_2$ scale).

Instance	Prange	Stern	BJMM	BothMay	MayOzerov	Complexity
TRIKE-1	168.66	147.38	147.72	147.60	147.60	134.05
TRIKE-2	208.44	185.71	186.02	185.95	185.95	171.78
TRIKE-3	238.39	214.79	215.06	215.03	215.03	200.36
TRIKE-5	306.97	281.72	281.90	281.78	281.78	266.61
TRIKE-7	442.98	415.44	414.98	414.90	414.90	398.81
TRIKE-9	578.22	549.05	548.21	548.21	548.21	531.41

Table 10. Message recovery time complexity (in $\log_2$ scale).

Instance	Prange	Stern	BJMM	BothMay	MayOzerov	Complexity
TRIKE-1	158.90	138.78	139.16	138.89	138.89	132.11
TRIKE-2	196.93	175.29	175.16	174.83	174.82	167.86
TRIKE-3	231.15	208.41	208.08	207.80	207.80	200.58
TRIKE-5	297.54	273.11	272.63	272.42	272.42	264.86
TRIKE-7	435.03	408.17	406.97	406.81	406.81	398.77
TRIKE-9	564.68	536.19	534.67	534.61	534.60	526.21

Appendices

A Details of the Weak Key Rejection

As observed in [6,12], the four classes of weak keys identified for QC-MDPC code are all associated with anomalous distance-spectrum multiplicities within a circulant block or unusually large intersections between different circulant blocks. Such structural irregularities lead to a significantly larger number of 4-cycles than expected for a randomly generated key and consequently increase the probability of decoding failure.

To detect these weak structures efficiently, we adopt the unified rejection methodology based on counting intra-block and inter-block 4-cycles.

Consider first a circulant block h_0 . Let

$$\text{Spec}(h_0) = \{(\delta, \mu(\delta, h_0)) \mid \delta \in \{1, \dots, (r-1)/2\}\}$$

denote its distance spectrum, where $\mu(\delta, h_0)$ is the multiplicity of distance δ in h_0 . According to [13], the number of 4-cycles induced within the block h_0 is

$$r \cdot \sum_{\delta=1}^{(r-1)/2} \binom{\mu(\delta, h_0)}{2}.$$

If this quantity exceeds a prescribed threshold $r \cdot s$, the key (h_0, h_1, h_2) is classified as weak and rejected. The same test is applied independently to h_1 and h_2 .

For inter-block structures, consider the pair (h_0, h_1) . Define

$$c(i, h_0, h_1) = w_H(h_0 \star x^i h_1), \quad i \in \{0, \dots, r-1\}.$$

The corresponding number of inter-block 4-cycles is given by

$$r \cdot \sum_{i=0}^{r-1} \binom{c(i, h_0, h_1)}{2}.$$

If this value exceeds a second threshold $r \cdot s'$, the key (h_0, h_1, h_2) is classified as weak and rejected. The same criterion is applied to the pairs (h_1, h_2) and (h_2, h_0) .

The complete filtering procedure is summarized in Algorithm 9. During key generation, candidate secret keys are repeatedly generated and tested until a key passes all intra-block and inter-block checks.

We now evaluate the filtering effectiveness of this procedure. By selecting the reference thresholds presented in Table 11, the proportion of rejected secret keys is bounded by 0.4%. According to the analysis in Section 4, this introduces a security loss of at most 0.0043 bits, which does not compromise the overall security of TRIKE. In the following, we analyze the filtering effectiveness against the Type I, Type II, and Type III weak keys introduced in [6], as well as weak keys satisfying the gathering property detailed in [12].

Since the total number of 4-cycles is inherently a multiple of r , let s_0 denote the normalized number of 4-cycles, given by the total count divided by r .

A.1 Type I

We call a weak key of Type I with parameter f , a key $h = (h_0, h_1, h_2)$ such that

$$h_i = \phi_\delta(x^l(1 + x + \dots + x^{f-1}) + h'_i)$$

for some $i \in \{0, 1, 2\}$, $\delta \in \{1, \dots, (r-1)/2\}$ and $l \in \{0, \dots, r-1\}$, with $w_H(h'_i) = d - f$ and $w_H(h_i) = d$. The density of such weak keys is bounded by:

$$\tau_I(f) \leq \frac{3r(r-1)\binom{r-f}{d-f}}{2\binom{r}{d}}.$$

Algorithm 9: Weak key Test**Input:** Secret key (h_0, h_1, h_2) with indices idx_0, idx_1, idx_2 , thresholds s, s' .**Output:** 1 if a weak key is detected, and 0 otherwise.

<pre> 1: Initialize $cnt_{ij} = 0$ for $i \in \{0, 1, 2\}, j \in \{0, \dots, (r-1)/2\}$ 2: for $u = 0$ to 2 do 3: for $i = 1$ to $d-1$ do 4: for $j = 0$ to $i-1$ do 5: $\delta = d(idx_{ui}, idx_{uj})$ 6: $cnt_{u\delta} = cnt_{u\delta} + 1$ 7: end for 8: $tot = 0$ 9: for $j = 1$ to $(r-1)/2$ do 10: $tot = tot + \binom{cnt_{uj}}{2}$ 11: end for 12: if $tot > s$ then 13: return 1 14: end if 15: end for 16: end for 17: if $\text{inter-block-test}(h_0, h_1, s')$ or $\text{inter-block-test}(h_1, h_2, s')$ or $\text{inter-block-test}(h_2, h_0, s')$ then 18: return 1 19: end if 20: return 0 </pre>	<pre> inter-block-test(h_a, h_b, s') 1: $tot = 0$ 2: for $i = 0$ to $r-1$ do 3: $\mu = w_H(h_a \star x^i h_b)$ 4: $tot = tot + \binom{\mu}{2}$ 5: end for 6: if $tot > s'$ then 7: return 1 8: else 9: return 0 10: end if </pre>
---	---

Table 11. Reference thresholds of weak key rejection

	TRIKE-1	TRIKE-2	TRIKE-3	TRIKE-5	TRIKE-7	TRIKE-9
s	28	46	59	105	235	433
s'	50	83	102	172	405	737
Rejection Rate	0.38%	0.19%	0.11%	0.06%	0.06%	0.22%

To evaluate the filtering effectiveness against Type I weak keys, we first identify the minimum parameter f_0 such that all weak keys with $f \geq f_0$ are strictly and completely filtered out by the threshold. We then calculate the corresponding density of this specific class of weak keys. To guarantee the security of the keys that remain unfiltered (i.e., keys just below the boundary with parameter $f = f_0 - 1$), we conducted extensive decapsulation experiments to measure the empirical Decapsulation Failure Rate (DFR). For the parameter sets of TRIKE-1/2/3/5, no decoding failures were observed in 10^7 experiments, which corresponds to a DFR below $2^{-21.67}$ at a 95% confidence level. For TRIKE-7/9, no failures were observed in 10^6 experiments, ensuring a DFR below $2^{-18.35}$ at a 95% confidence level.

For the filtered keys, within the f_0 consecutive 1s, the structural overlaps guarantee that $\mu(1, h_i) \geq f_0$, $\mu(2, h_i) \geq f_0 - 1$, and so forth. Consequently, s_0 satisfies the lower bound:

$$s_0 \geq \sum_{i=1}^{f_0} \binom{f_0 - i + 1}{2}.$$

Based on this bound, the numerical evaluation presented in Table 12 confirms that all Type I weak keys satisfying these conditions are completely filtered out.

A.2 Type II

We call a weak key of Type II with parameter m , a key $h = (h_0, h_1, h_2)$ such that $\mu(\delta, h_i) = m$ for some $i \in \{0, 1, 2\}$ and some distance $\delta \in \{1, \dots, (r-1)/2\}$. The density of such weak keys is

Table 12. Evaluation of filtering effectiveness against Type I weak keys

Instance	f_0	Density	s_0^{LB}	s	Miss rate
TRIKE-1	6	$2^{-25.07}$	35	28	0
TRIKE-2	7	$2^{-34.07}$	56	46	0
TRIKE-3	8	$2^{-44.16}$	84	59	0
TRIKE-5	9	$2^{-54.15}$	120	105	0
TRIKE-7	12	$2^{-85.00}$	286	235	0
TRIKE-9	14	$2^{-65.49}$	455	433	0

bounded by:

$$\tau_{II}(m) \leq \frac{3r(r-1)\binom{d-1}{d-m-1}\binom{r-d-1}{d-m-1}}{2(d-m)\binom{r}{d}}.$$

Following the same evaluation methodology as Type I, we first identify the minimum parameter m_0 required for complete filtering by the threshold and then calculate its corresponding density. To verify the security of the boundary keys that remain unfiltered (where $m = m_0 - 1$), we performed the identical decapsulation testing configuration (10^7 experiments for TRIKE-1/2/3/5 and 10^6 for TRIKE-7/9). With zero decoding failures observed, the empirical DFR similarly satisfies the upper bounds of $2^{-21.67}$ and $2^{-18.35}$ at a 95% confidence level, respectively.

For the filtered keys, we now evaluate the minimal lower bound of s_0 using the guaranteed contribution derived strictly from the m_0 pairs of positions $(u, u + \delta_0 \bmod r)$ from $\mu(\delta_0, h_i) \geq m_0$.

Let $G = (V, E)$ be a directed graph where the vertex set V represents the set of unique index position u or $(u + \delta_0 \bmod r)$ from the m_0 pairs. We construct the edge set E by adding a directed edge from u to $(u + \delta_0 \bmod r)$ for each of the overlapping pairs.

Because the shift δ_0 is constant across all pairs, the maximum in-degree and out-degree of any vertex in G is at most 1. Given that r is a prime number and $r \gg m_0$, G cannot contain any cycles. Therefore, G decomposes uniquely into a set of k disjoint paths $P_1, P_2, \dots, P_k$ with lengths $l_1, l_2, \dots, l_k$ respectively, such that $\sum_{i=1}^k l_i = m_0$.

In the subsequent analysis, we treat the relative shifts generated within and across paths as distinct. If two structural configurations yield identical relative shifts (e.g., $\delta_1 = \delta_2$), causing positional overlaps of size a and b to coincide, the combined intersection weight becomes at least $a + b$. We have $\binom{a+b}{2} > \binom{a}{2} + \binom{b}{2}$. Therefore, any coincidental shift collisions will strictly increase the final sum. This property allows us to safely compute a conservative lower bound for s_0 by treating all derived shift pairs as independent.

To evaluate s_0 , we categorize the localized intersection metrics into three independent structural components:

1. Base Shift Contribution: The base shift δ yields $c(\delta, h_i, h_j) = m_0$, which contributes $\binom{m_0}{2}$.
2. Intra-Path Contribution (I_{intra}): For any path P_i of length $l_i \geq 2$, higher-order internal shifts (i.e., $2\delta, 3\delta, \dots$) yield sub-path overlaps. Summing these internal binomials gives:

$$I_{\text{intra}} = \sum_{i=1}^k \sum_{j=1}^{l_i-1} \binom{l_i-j}{2}.$$

3. Inter-Path Contribution (I_{inter}): For any two distinct paths P_i and P_j , there are $l_i l_j$ pairs of edges. Each pair forms a unique relative shift that guarantees an intersection weight of at least 2, contributing $\binom{2}{2} = 1$. Summing over all distinct pairs of paths yields:

$$I_{\text{inter}} = \sum_{1 \leq i < j \leq k} l_i l_j.$$

The total metric is bounded by:

$$s_0 \geq \binom{m_0}{2} + I_{\text{intra}} + I_{\text{inter}}.$$

The minimum value of this expression can be computed using dynamic programming. Utilizing this lower bound, the numerical evaluation presented in Table 13 demonstrates that all Type II weak keys satisfying these conditions are completely filtered out.

Table 13. Evaluation of filtering effectiveness against Type II weak keys

Instance	m_0	Density	s_0^{LB}	s	Miss rate
TRIKE-1	7	$2^{-28.06}$	38	28	0
TRIKE-2	8	$2^{-33.07}$	51	46	0
TRIKE-3	9	$2^{-40.00}$	66	59	0
TRIKE-5	12	$2^{-59.87}$	124	105	0
TRIKE-7	17	$2^{-93.91}$	261	235	0
TRIKE-9	22	$2^{-130.02}$	448	433	0

A.3 Type III

We define a Type III weak key with parameter m as a key $h = (h_0, h_1, h_2)$ such that $w_H(h_i \star x^\delta h_j) = m$ for some distinct $i, j \in \{0, 1, 2\}$ and shift $\delta \in \{0, \dots, r-1\}$. The density of such weak keys is bounded by:

$$\tau_{III}(m) \leq \frac{3r \binom{d}{m} \binom{r-d}{d-m}}{\binom{r}{d}}.$$

Following the same evaluation methodology as the previous types, we first identify the minimum parameter m_0 required for complete filtering by the threshold and then calculate its corresponding density. To verify the security of the boundary keys that remain unfiltered (where $m = m_0 - 1$), we performed the identical decapsulation testing configuration (10^7 experiments for TRIKE-1/2/3/5 and 10^6 for TRIKE-7/9). With zero decoding failures observed, the empirical DFR similarly satisfies the upper bounds of $2^{-21.67}$ and $2^{-18.35}$ at a 95% confidence level, respectively.

For the filtered keys ($m \geq m_0$), we evaluate the minimal lower bound of s_0 by modeling the m_0 overlapping position pairs $(u, u + \delta \bmod r)$ as a directed graph $G = (V, E)$, where each edge represents a valid overlap between $\text{supp}(h_i)$ and $\text{supp}(x^\delta h_j)$. Since the shift δ is constant across all pairs, the maximum in-degree and out-degree of any vertex in G is at most 1. Given that r is a prime number and $r \gg m_0$, G cannot contain any cycles. Therefore, G uniquely decomposes into a set of k disjoint paths $P_1, P_2, \dots, P_k$. We define l_i as the length (i.e., the number of edges) of the i -th path P_i , which naturally satisfies the conservation constraint $\sum_{i=1}^k l_i = m_0$. A lower bound for s_0 is given by

$$s_0 \geq \binom{m_0}{2} + I_{\text{intra}} + I_{\text{inter}}$$

where

$$I_{\text{intra}} = \sum_{i=1}^k \sum_{j=1}^{l_i-1} \binom{l_i-j}{2} \quad \text{and} \quad I_{\text{inter}} = \sum_{1 \leq i < j \leq k} l_i l_j.$$

The minimum value of this expression can be computed using dynamic programming. Utilizing this lower bound, the numerical evaluation presented in Table 14 demonstrates that all Type III weak keys satisfying these conditions are completely filtered out.

A.4 Gathering Property

We call a weak key satisfying the (r_s, ϵ) -gathering property, a key $h = (h_0, h_1, h_2)$ such that

$$h_i = \phi_\delta(x^l h'_i + h''_i)$$

Table 14. Evaluation of filtering effectiveness against Type III weak keys

Instance	m_0	Density	s_0^{LB}	s	Miss rate
TRIKE-1	8	$2^{-34.31}$	51	50	0
TRIKE-2	10	$2^{-47.12}$	84	83	0
TRIKE-3	11	$2^{-54.36}$	103	102	0
TRIKE-5	14	$2^{-74.60}$	173	172	0
TRIKE-7	21	$2^{-93.91}$	406	405	0
TRIKE-9	28	$2^{-130.02}$	738	737	0

for some $i \in \{0, 1, 2\}$, $\delta \in \{1, \dots, (r-1)/2\}$ and $l \in \{0, \dots, r-1\}$, with $\deg(h'_i) \leq r_s$, $w_H(h'_i) = d - \epsilon$, $w_H(h''_i) = \epsilon$ and $w_H(h_i) = d$.

The density of such weak keys is bounded by:

$$\tau_{gather} \leq \frac{3r(r-1) \sum_{i=d-\epsilon}^{r_s} \binom{i-2}{d-2-\epsilon} \binom{r-i}{\epsilon} \binom{r}{d}}{2^{\binom{r}{d}}}.$$

Practical results indicate that for a given density, the filtering effectiveness for cases where $\epsilon > 0$ is no worse than when $\epsilon = 0$. Therefore, in the following analysis, we restrict our focus exclusively to the $\epsilon = 0$ case.

To validate filtering effectiveness, experimental estimation of DFR and theoretical analyzes of miss rate are combined for a resulting security estimation under weak key rejection.

The DFR estimation without weak key rejection is obtained by repeatedly generating weak keys satisfying the gathering property at a specific density, executing the TRIKE bit-flipping decoder for decoding, and statistically counting the decoding failures to determine the density of weak keys. The theoretical analysis of the miss rate is established by modeling the distribution of the distance spectrum for components h_i that satisfy the gathering property. By estimating the distribution of the distance collision count within h_i (which directly corresponds to the distribution of s_0), we derive the estimated miss rate as $\rho = \Pr[s_0 \leq s]$.

The corresponding experimental results and comparisons are recorded in Table 15.

Table 15. Evaluation of filtering effectiveness against weak keys with gathering property

Instance	s	r_{s0}	$\log_2(\text{Density})$	$\log_2(\rho)$	$\log_2(\text{DFR})$	$\log_2(\text{Sum})$
TRIKE-1	28	700	-82.87	-73.40	-19.79	-176.07
		600	-88.78	-95.33	-16.50	-200.61
		500	-95.79	-127.41	-13.13	-236.33
TRIKE-2	46	1000	-115.87	-168.15	-20.25	-304.27
		900	-121.14	-198.94	-17.06	-337.14
		800	-127.04	-238.38	-14.53	-379.94
TRIKE-3	59	1300	-143.74	-264.66	-20.08	-428.49
		1200	-148.44	-299.26	-18.08	-465.78
		1100	-153.55	-340.82	-16.11	-510.48
TRIKE-5	105	1900	-207.38	-664.91	-19.67	-891.96
		1800	-211.66	-718.72	-17.47	-947.85
		1700	-216.18	-779.37	-15.13	-1010.69
TRIKE-7	235	4700	-296.93	-1374.33	-17.12	-1688.38
		4500	-302.12	-1464.88	-16.12	-1783.13
		4300	-307.55	-1564.56	-14.98	-1887.09
TRIKE-9	433	7900	-400.54	-2677.05	-14.60	-3092.18
		7700	-404.64	-2778.87	-14.47	-3197.98
		7500	-408.84	-2886.56	-13.70	-3309.10

References

1. Carlos Aguilar Melchor, Nicolas Aragon, Slim Bettaieb, Loïc Bidoux, Olivier Blazy, Jean-Christophe Deneuville, Philippe Gaborit, Edoardo Persichetti, Gilles Zémor, Jurjen Bos, Arnaud Dion, Jérôme Lacan, Jean-Marc Robert, Pascal Véron, Paulo L. Barreto, Santosh Ghosh, Shay Gueron, Tim Gúneysu, Rafael Misoczki, Jan Richter-Brokmann, Nicolas Sendrier, Jean-Pierre Tillich, and Valentin Vasseur. Hamming quasi-cyclic (hqc). Technical report, National Institute of Standards and Technology, 2025. available at https://pqc-hqc.org/doc/hqc_specifications_2025_08_22.pdf.
2. Nicolas Aragon, Paulo Barreto, Slim Bettaieb, Loïc Bidoux, Olivier Blazy, Jean-Christophe Deneuville, Phillippe Gaborit, Santosh Ghosh, Shay Gueron, Tim Gúneysu, Carlos Aguilar Melchor, Rafael Misoczki, Edoardo Persichetti, Jan Richter-Brokmann, Nicolas Sendrier, Jean-Pierre Tillich, Valentin Vasseur, and Gilles Zémor. Bike: bit flipping key encapsulation. Technical report, National Institute of Standards and Technology, 2024. available at https://bikesuite.org/files/v5.2/BIKE_Spec.2024.10.10.1.pdf.
3. National institute of standards and technology: Post-quantum cryptography project. <https://csrc.nist.gov/projects/post-quantum-cryptography>, 2016.
4. Dennis Hofheinz, Kathrin Hövelmanns, and Eike Kiltz. A modular analysis of the Fujisaki-Okamoto transformation. pages 341–371, 2017.
5. Julia Chaulet. *Étude de cryptosystèmes à clé publique basés sur les codes MDPC quasi-cycliques*. PhD thesis, Paris 6, 2017.
6. Valentin Vasseur. *Post-quantum cryptography: a study of the decoding of QC-MDPC codes*. PhD thesis, Université de Paris, 2021.
7. Notice on the call for proposals for next-generation public-key cryptographic algorithms. Technical report, Institute of Commercial Cryptography Standards, 2025.
8. Martin R. Albrecht, Daniel J. Bernstein, Tung Chou, Carlos Cid, Jan Gilcher, Tanja Lange, Varun Maram, Ingo von Maurich, Rafael Misoczki, Ruben Niederhagen, Kenneth G. Paterson, Edoardo Persichetti, Christiane Peters, Peter Schwabe, Nicolas Sendrier, Jakub Szefer, Cen Jung Tjhai, Martin Tomlinson, and Wen Wang. Classic McEliece. Technical report, National Institute of Standards and Technology, 2022. available at <https://csrc.nist.gov/projects/post-quantum-cryptography/round-4-submissions>.
9. Nir Drucker, Shay Gueron, and Dusan Kostic. Fast polynomial inversion for post quantum QC-MDPC cryptography. *Information and Computation*, 281:104799, 2021.
10. Nir Drucker, Shay Gueron, and Vlad Krasnov. Fast multiplication of binary polynomials with the forthcoming vectorized vpclmulqdq instruction. In *2018 IEEE 25th Symposium on Computer Arithmetic (ARITH)*, pages 115–119, 2018.
11. Abdelaziz Amraoui, Andrea Montanari, Tom Richardson, and Rüdiger Urbanke. Finite-length scaling for iteratively decoded ldpc ensembles. *IEEE Transactions on Information Theory*, 55(2):473–498, 2009.
12. Tianrui Wang, Anyu Wang, and Xiaoyun Wang. Exploring decryption failures of BIKE: New class of weak keys and key recovery attacks. pages 70–100, 2023.
13. Gretchen L. Matthews and Emily McMillon. A combinatorial approach to avoiding weak keys in the BIKE cryptosystem. 93(8):3413–3436, 2025.
14. Andre Esser, Javier A. Verbel, Floyd Zweyding, and Emanuele Bellini. {SoK: CryptographicEstimators - a Software Library for Cryptographic Hardness Estimation}. In *AsiaCCS*. {ACM}, 2024.
15. Eugene Prange. The use of information sets in decoding cyclic codes. *IRE Transactions on Information Theory*, 8(5):5–9, 1962.
16. Daniel J. Bernstein, Tanja Lange, and Christiane Peters. Attacking and defending the McEliece cryptosystem. pages 31–46, 2008.
17. Christiane Peters. Information-set decoding for linear codes over F_q . pages 81–94, 2010.
18. Jacques Stern. A method for finding codewords of small weight. In *Coding Theory and Applications: 3rd International Colloquium Toulon, France, November 2–4, 1988 Proceedings 3*, pages 106–113. Springer, 1989.
19. Anja Becker, Antoine Joux, Alexander May, and Alexander Meurer. Decoding random binary linear codes in $2^{n/20}$: How $1+1=0$ improves information set decoding. In *Advances in Cryptology—EUROCRYPT 2012: 31st Annual International Conference on the Theory and Applications of Cryptographic Techniques, Cambridge, UK, April 15–19, 2012. Proceedings 31*, pages 520–536. Springer, 2012.
20. Alexander May, Alexander Meurer, and Enrico Thomae. Decoding random linear codes in $\tilde{O}(2^{0.054n})$. In *Advances in Cryptology—ASIACRYPT 2011: 17th International Conference on the Theory and Application of Cryptology and Information Security, Seoul, South Korea, December 4–8, 2011. Proceedings 17*, pages 107–124. Springer, 2011.

21. Andre Esser and Floyd Zveydinger. New time-memory trade-offs for subset sum – improving ISD in theory and practice. In Carmit Hazay and Martijn Stam, editors, *2023, Part V*, pages 360–390, 2023.
22. Leif Both and Alexander May. Decoding linear codes with high error rate and its impact for LPN security. pages 25–46, 2018.
23. Alexander May and Ilya Ozerov. On computing nearest neighbors with applications to decoding of binary linear codes. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 203–228. Springer, 2015.